



Bases de datos objeto-relacionales y orientadas a objetos.



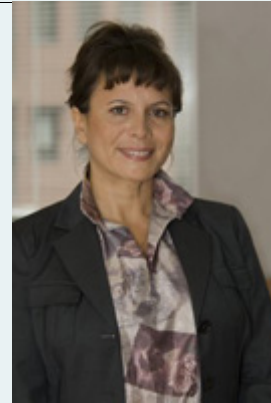
Caso práctico

Esta mañana **Juan, María y Ana** han quedado a primera hora en el despacho de **Ada**. Según les comentó ayer **Ada**, antes de cerrar, urge comenzar a planificar un proyecto nuevo que ha llegado a la empresa. Se trata de desarrollar una aplicación informática para una empresa de la industria de la madera. La empresa se dedica a la fabricación de mobiliario de diseño, así como a su distribución y venta.

La aplicación que desarrollen debe gestionar una gran volumen de piezas diferentes, piezas de chapa y madera, permitiendo diseñarlas desde la forma más simple a la más compleja. También, debe permitir desplegar y gestionar directamente la evolución de la pieza añadiendo pliegues, cortes, deformaciones, etc. Por descontado, debe gestionar también el almacenamiento de todas esas piezas y los productos finales.

Ada y Juan tienen claro que en esta aplicación no tiene cabida el uso de bases de datos relacionales, ya que habrá que almacenar un gran volumen de datos diferentes, con propiedades y comportamientos que no son fijos, y con complejas relaciones entre ellos. Será necesario un tipo de bases de datos más avanzado, posiblemente orientadas a objetos u objeto-relacionales, sin las limitaciones que presentan las bases de datos relacionales, entre ellas, que solo son capaces de manejar estructuras muy simples (filas y columnas).

Ana ha estado algo callada durante la reunión, sabe que ella va a echar una mano en este proyecto junto a **Antonio**, y necesita repasar muchísimo sobre bases de datos avanzadas. Lo primero que ha decidido hacer es coger sus apuntes del ciclo de DAM e ir echando un vistazo a las características de las bases de datos orientadas a objetos y objeto-relacionales.



Ministerio de Educación (Uso educativo nc)

Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

[Aviso Legal](#) 

1.- Introducción.

Las Bases de Datos Relacionales (BDR) son ideales para aplicaciones tradicionales que soportan tareas administrativas, y que trabajan con datos de estructuras simples y poco cambiantes, incluso cuando la aplicación pueda estar desarrollada en un lenguaje OO y sea necesario un Mapeo Objeto Relacional (ORM)



Q Isabel M. Cruz Granados (Uso

educativo no)

Pero cuando la aplicación requiere otras necesidades, como por ejemplo, soporte multimedia, almacenar objetos muy cambiantes y complejos en estructura y relaciones, este tipo de base de datos no son las más adecuadas. Recuerda, que si queremos representar un objeto y su relaciones en una BDR esto implica que:

- Los objetos deben ser descompuestos en diferentes tablas.
- A mayor complejidad, mayor número de tablas, de manera que se requieren muchos enlaces (joins) para recuperar un objeto, lo cual disminuye dramáticamente el rendimiento.

Las **Bases de Datos Orientadas a Objetos (BD OO)** o Bases de Objetos se integran directamente y sin problemas con las aplicaciones desarrolladas en lenguajes orientados a objetos, ya que **soportan un modelo de objetos puro** y son ideales para almacenar y recuperar datos complejos permitiendo a los usuarios su [navegación](#)  directa (sin un mapeo entre distintas representaciones).

Las Bases de Objetos aparecieron a finales de los años 80 motivadas fundamentalmente por dos razones:

- Las necesidades de los lenguajes de Bases de Datos Orientadas a Objetos (POO), como la necesidad de persistir objetos.
- Las limitaciones de las bases de datos relacionales, como el hecho de que sólo manejan estructuras muy simples (tablas) y tienen poca riqueza semántica



Q Agustin Diaz (CC BY-SA)

Pero como las BD OO no terminaban de asentarse, debido fundamentalmente a la inexistencia de un estándar, y las BDR gozaban y gozan en la actualidad de una gran aceptación, experiencia y difusión, debido fundamentalmente a su gran robustez y al lenguaje SQL, los fabricantes de bases de datos comenzaron a implementar nuevas funcionalidades orientadas a objetos en las BDR existentes. Así surgieron, las bases de datos objeto-relacionales.

Las **Bases de Datos Objeto-Relacionales (BDOR)** son bases de datos relacionales que han evolucionado hacia una base de datos más extensa y compleja, incorporando conceptos del modelo orientado a objetos. Pero en

estas bases de datos **aún existe un mapeo de objetos subyacente**, que es costoso y poco flexible, cuando los objetos y sus interacciones son complejos.

2.- Características de las bases de datos orientadas a objetos.



Caso práctico



Q Ministerio de Educación (Uso educativo [nc](#))

el sistema de almacenamiento óptimo para esta aplicación. Ya sabes que no podemos fallar. ¡Somos los mejores! —y le sonríe.

Ana se ha llevado a la empresa sus apuntes sobre bases de datos y acceso a datos del ciclo formativo, y ha empezado a repasar con **Juan** las características de las Bases de Objetos.

—¡Pero claro!, —le dice Juan a Ana— habrá que detenerse en valorar tanto las posibles ventajas como los inconvenientes de estas bases de datos, ya que tenemos que elegir

En una BDOO, los datos se almacenan como objetos. Un **objeto** es, al igual que en POO, una entidad que se puede identificar unívocamente y que describe tanto el [estado](#)  como el [comportamiento](#)  de una entidad del 'mundo real'. El estado de un objeto se describe mediante atributos y su comportamiento es definido mediante procedimientos o métodos.

Entonces, ¿a qué equivalen las entidades, ocurrencias de entidades y relaciones del modelo relacional? Las entidades son las clases, las ocurrencias de entidad son objetos creados desde las clases, las relaciones se mantienen por medio de inclusión lógica, y no existen claves primarias, los objetos tienen un identificador.



Q Pixabay (CC0)

La principal característica de las BDOO es que **soportan un modelo de objetos puro y que el lenguaje de programación y el esquema de la base de datos utilizan las mismas definiciones de tipos**.

Otras características importantes de las BDOO son las siguientes:

- **Soportan las características propias de la Orientación a Objetos** como agregación, encapsulamiento, polimorfismo y herencia. La herencia se mantiene en la propia base de datos.
- **Identificador de objeto (OID)**. Cada objeto tiene un identificador, generado por el sistema, que es único para cada objeto, lo que supone que cada vez que se necesite modificar un objeto, habrá que recuperarlo de la base de datos, hacer los cambios y almacenarlo nuevamente. Los OID son independientes del contenido del objeto, esto es, si cambia su información, el objeto sigue teniendo el mismo OID. Dos objetos serán equivalentes si tienen la misma información pero diferentes OID.

- **Jerarquía y extensión de tipos.** Se pueden definir nuevos tipos basándose en otros tipos predefinidos, cargándolos en una jerarquía de tipos (o jerarquía de clases).
- **Objetos complejos.** Los objetos pueden tener una estructura de objeto de complejidad arbitraria, a fin de contener toda la información necesaria que describe el objeto.
- **Acceso navegacional de datos.** Cuando los datos se almacenan en una estructura de red densa y probablemente con una estructura de diferentes niveles de profundidad, el acceso a datos se hace principalmente navegando la estructura de objetos y se expresa de forma natural utilizando las construcciones nativas del lenguaje, sin necesidad de uniones o joins típicas en las BDR.
- **Gestión de versiones.** El mismo objeto puede estar representado por múltiples versiones. Muchas aplicaciones de bases de datos que usan orientación a objetos requieren la existencia de varias versiones del mismo objeto, ya que si estando la aplicación en funcionamiento es necesario modificar alguno de sus módulos, el diseñador deberá crear una nueva versión de cada uno de ellos para efectuar cambios.



Autoevaluación

Señala las opciones correctas. Las bases de datos orientadas a objetos:

- ☐ Soportan conceptos de orientación a objetos como la herencia.
- ☐ Permiten la manipulación navegacional.
- ☐ Tienen el mismo tipo de problemas que las relacionales para gestionar objetos complejos.
- ☐ Cada objeto posee un identificador de objeto.

Mostrar retroalimentación

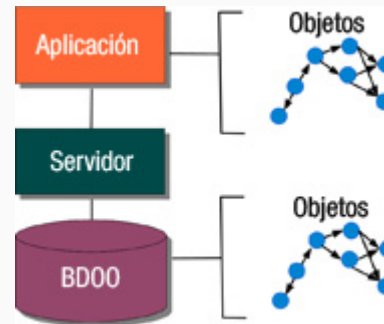
Solución

1. Correcto (#answer-6_58)
2. Correcto (#answer-6_85)
3. Incorrecto (#answer-6_87)
4. Correcto (#answer-6_89)

2.1.- Ventajas e inconvenientes.

El uso de una BD00 puede ser ventajoso frente a una BDR relacional si nuestra aplicación requiere alguno de estos elementos :

- Un gran número de tipos de datos diferentes.
- Un gran número de relaciones entre los objetos.
- Objetos con comportamientos complejos.



Q Isabel M. Cruz Granados (Uso educativo

nc)

Una de las principales ventajas de los sistemas de bases de datos orientados a objetos es la **transparencia**, (manipulación directa de datos utilizando un entorno de programación basado en objetos), por lo que el programador, solo se debe preocupar de los objetos de su aplicación, en lugar de cómo los debe almacenar y recuperar de un medio físico.

Otras ventajas de un sistema de bases de datos orientado a objetos son las siguientes:

- **Gran capacidad de modelado.** El modelado de datos orientado a objetos permite modelar el 'mundo real' de una manera óptima gracias al encapsulamiento y la herencia.
- **Flexibilidad.** Permiten una estructura cambiante con solo añadir subclases.
- **Soporte para el manejo de objetos complejos.** Manipula de forma rápida y ágil objetos complejos, ya que la estructura de la base de datos está dada por referencias (apuntadores lógicos) entre objetos.
- **Alta velocidad de procesamiento.** Como el resultado de las consultas son objetos, no hay que reensamblar los objetos cada vez que se accede a la base de objetos
- **Extensibilidad.** Se pueden construir nuevos tipos de datos a partir de los ya existentes, agrupar propiedades comunes de diversas clases e incluirlas en una superclase, lo que reduce la redundancia.
- **Mejora los costes de desarrollo,** ya que es posible la reutilización de código, una de las características de los lenguajes de programación orientados a objetos.
- **Facilitar el control de acceso y concurrencia,** puesto que se puede bloquear a ciertos objetos, incluso en una jerarquía completa de objetos.
- Funcionan de forma **eficiente en entornos cliente/servidor y arquitecturas distribuidas.**

Pero aunque los sistemas de bases de datos orientados a objetos pueden proporcionar soluciones apropiadas para muchos tipos de aplicaciones avanzadas de bases de datos, también tienen sus **desventajas**. Éstas son las

siguientes:

- **Carencia de un modelo de datos universal.** No hay ningún modelo de datos aceptado universalmente, y la mayor parte de los modelos carecen de una base teórica.
- **Falta de estándares.** Existe una carencia de estándares general para los sistemas de BDOO.
- **Complejidad.** La estructura de una BDOO es más compleja y difícil de entender que la de una BDR.
- **Competencia de otros modelos.** Las bases de datos relacionales y objeto-relacionales están muy asentadas y extendidas, siendo un duro competidor.
- **Difícil optimización de consultas.** La optimización de consultas requiere una comprensión de la implementación de los objetos, para poder acceder a la base de datos de manera eficiente. Sin embargo, esto compromete el concepto de encapsulación.



Citas para pensar

"Cada día sabemos más y entendemos menos".

Albert Einstein



Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

En una BDOO el resultado de una consulta son objetos, por lo que no es necesario reensamblar los objetos cada vez que se accede a la base de datos.

☐ Verdadero ☐ Falso

Verdadero

Es verdadero, precisamente ésta es una de las ventajas de estos sistemas de almacenamiento.

3.- Gestores de bases de datos orientadas a objetos.



Caso práctico

Esta mañana, **Juan** ha preparado una serie de sistemas de bases de objetos diferentes, para instalarlos e ir probando algunas de sus características.

Juan le pregunta a **Ana** —¿Sabes qué al contrario de las bases de datos relacionales, los gestores de bases de datos orientados a objetos pueden llegar a ser muy diferentes entre sí?

A lo que **Ana** responde —Sí, en clase estuvimos trabajando con diferentes sistemas, precisamente para dar fe de ello, y..., quiero recordar que había una que tenía el nombre de un pintor francés. ¿Matisse?

—Efectivamente —dice **Juan**—, pero hoy he pensado en ver la base de objetos Db4o, que es la que actualmente utiliza el sistema de trenes español AVE. ¿Lo sabías?



Q Ministerio de Educación (Uso educativo nc)

Un **Sistema Gestor de Bases de Datos Orientada a Objetos** (SGBDOO) y en inglés ODBMS, Object Databases Management System) es un software específico, dedicado a servir de [interfaz](#) entre la base de objetos, el usuario y las aplicaciones que la utilizan. Un SGBDOO incorpora el paradigma de Orientación a Objetos y permite el almacenamiento de objetos en soporte secundario:

- Por ser SGBD debe incluir mecanismos para optimizar el acceso, gestionar el control de concurrencia, la seguridad y la gestión de usuarios, así como facilitar la consulta y recuperación ante fallos.
- Por ser OO incorpora características de identidad, encapsulación, herencia, polimorfismo y control de tipos.

Cuando aparecieron las bases de datos orientadas a objetos, un grupo formado por desarrolladores y usuarios de bases de objetos, denominado ODMG (Object-Oriented Database Management Group), propuso un estándar que se conoce como estándar **ODMG-93** y que se ha ido revisando con el tiempo, pero que en realidad **no ha tenido mucho éxito**, aunque es un punto de partida.



Q Isabel M. Cruz Granados (Uso educativo nc)



Debes conocer



Desde el siguiente enlace te puedes descargar un resumen de las características del estándar ODMG.

[Resumen del estándar ODMG](#) 

¿Qué estrategias o enfoques se siguen para el desarrollo de SGBDOO? Básicamente, las siguientes:

- Ampliar un lenguaje de programación OO existente con capacidades de BD (Ejemplo: GemStone).
- Proporcionar bibliotecas de clases con las capacidades tradicionales de las bases de datos, como persistencia, transacciones, concurrencia, etc., (Ejemplo: ObjectStore y Versant).
- Ampliar un lenguaje de BD con capacidades OO, caso de SQL 2003 y Object SQL (OQL, propuesto por ODMG).

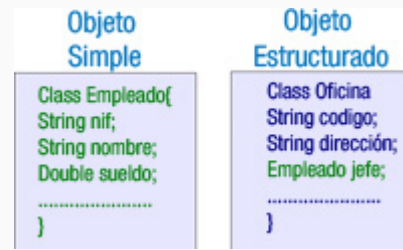
Tal y como estarás pensando, la carencia de un estándar real hace difícil el soporte para la [portabilidad](#)  de aplicaciones y su [interoperabilidad](#) , y es en parte por ello, que a diferencia de las bases de datos relacionales donde hay muchos productos donde elegir, la variedad de sistemas de bases de datos orientadas a objetos es mucho menor. En la actualidad hay diferentes productos de este tipo, tanto con licencia libre como propietaria.

A continuación te indicamos algunos **ejemplos de SGBDOO**:

- **Db4o de Versant.** Es una BDOO Open Source para Java y .NET. Se distribuye bajo licencia [GPL](#).
- **Matisse.** Es un SGBDOO basado en la especificación ODMG, proporciona lenguajes para definición y manipulación de objetos, así como interfaces de programación para C, C++, Eiffel y Java.
- **ObjectDB.** Es una BDOO que ofrece soporte para Java, C++, y Python entre otros lenguajes. No es un producto libre, aunque ofrecen versiones de prueba durante un periodo determinado.
- **EyeDB.** Es un SGBDOO basado en la especificación ODMG, proporciona lenguajes para definición y manipulación de objetos, e interfaces de programación para C++ y Java. Se distribuye bajo licencia [GNU](#) y es software libre.
- Neodatis, ObjectStore y GemStone. Son otros SGBDOO.

3.1.- Objetos simples y objetos estructurados.

En las BDOO los objetos se encuentran interrelacionados por referencias entre ellos de manera similar a como los objetos se referencian entre sí en memoria.



Q Isabel M. Cruz Granados (Uso educativo no)

Un **objeto de tipo simple** u objeto simple es aquel que no contiene a otros objetos y por tanto posee una estructura de un solo nivel de profundidad en este sentido.

Un **objeto de tipo estructurado** u objeto estructurado incluye entre sus componentes a otros objetos y se define aplicando los constructores de tipos disponibles por el SGBDOO recursivamente a varios niveles de profundidad.

Entre un objeto y sus componentes de cada nivel, existen dos tipos de referencia:

- **Referencia de propiedad.** Se aplica cuando los componentes de un objeto se encapsulan dentro del propio objeto y se consideran, por tanto, parte de ese objeto. **Relación es-parte-de.** No necesitan tener identificadores de objeto y sólo los métodos de ese objeto pueden acceder a ellos. Desaparecen si el propio objeto se elimina.
- **Referencia de asociación.** Se aplica cuando entre los componentes del objeto estructurado existen objetos independientes, pero es posible hacer referencia a ellos desde el objeto estructurado. **Relación está-asociado-con.** Cuando un objeto estructurado tiene que acceder a sus componentes referenciados, lo hace invocando los métodos apropiados de los componentes, ya que no están encapsulados dentro del objeto estructurado.

Por ejemplo, si observas la figura superior derecha, en un objeto tipo Oficina los componentes `codigo` y `dirección` son parte del objeto, mientras que el componente `jefe`, es un objeto independiente que está asociado con el objeto oficina.

Entonces, ¿las referencias de asociación son como las relaciones del modelo relacional? Así es:

- La referencia de asociación representa las relaciones o interrelaciones entre objetos independientes, dando la posibilidad de que los objetos puedan detectarse mutuamente en una o dos direcciones, (lo que en el modelo relacional representamos mediante claves ajenas o foráneas que provienen de relaciones uno a uno, uno a muchos y muchos a muchos).
- Una relación uno a muchos se representa mediante un objeto tipo colección (`List`, `Set`, etc.). En una BDOO la colección se maneja como cualquier otro objeto (aunque potencialmente profundo) y normalmente se podrá recuperar y almacenar el objeto padre junto con la colección asociada y su contenido en una sola llamada.

Además, un objeto miembro referenciado puede ser referenciado por más de un objeto estructurado y, no se elimina automáticamente cuando se elimina el objeto del nivel superior.



Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

Un objeto estructurado es aquel que contiene a otros objetos.

☐ Verdadero ☐ Falso

Verdadero

Es verdadero, efectivamente es estructurado cuando contiene a otro u otros objetos y en diferentes niveles de profundidad.

3.2.- Instalación del gestor de objetos Db4o.

Java, disponible para Java y .Net, y utilizada en la actualidad por diversas compañías para el desarrollo de aplicaciones de dispositivos móviles, dispositivos médicos y biotecnología, aplicaciones web, software enlatado y aplicaciones en tiempo real.

Tres **características importantes de Db4o** son las siguientes:



Q Luis Perez (CC BY-NC-SA)

- El modelo de clases es el propio esquema de la base de datos, por lo que se elimina el proceso de diseño, implementación y mantenimiento de la base de datos.
- Está diseñada bajo la estrategia de proporcionar bibliotecas de clases con las capacidades tradicionales de las bases de datos, y con el objetivo de cero administración.
- Puede trabajar como **base de datos embebida**, lo que significa que se puede distribuir con la aplicación, y solo la aplicación que lanza la base de datos embebida puede acceder a ella, siendo ésta invisible para el usuario final.

Una de las ventajas de db4o es que es un simple fichero .jar distribuible con cualquier aplicación sin necesidad de tener que instalar nada. Tampoco necesita drivers de tipo JDBC o similar.

La instalación de db4o consistirá en:

- Instalar el motor de base de datos, que son las clases necesarias para hacer que funcione la API en toda su extensión.
- Instalar alguna aplicación para visualizar los datos con los que se está trabajando. Esto último es necesario, pues en otro caso se trabajaría a ciegas con los datos.

En el siguiente vídeo puedes ver como incluir las dependencias necesarias en Apache Netbeans para trabajar con el SGBD OO DB4o.

Puedes descargarte la librería pulsando en este [enlace](#).

<https://www.youtube.com/embed/0L1N9r1qSFU>

4.- El API de la base de objetos.



Caso práctico



Q Ministerio de Educación (Uso educativo nc)

un ardua sesión de trabajo, familiarizarse con el API de la base de objetos Db4o.

Ana no puede negar su entusiasmo con el hecho de participar en el desarrollo del nuevo proyecto. Cuando estudió Acceso a Datos en el Ciclo Formativo, nunca pensó que llegaría a trabajar tan de cerca con las bases de datos avanzadas.

Ha pasado en el descanso a ver a su amigo **Antonio** y ponerlo al día de los avances que va realizando junto a **Juan**. Hoy le espera

Todos los SGBD OO, independientemente de su estrategia de diseño, proporcionan un API (Interfaz de Programación de Aplicaciones), más o menos extenso, disponible para ciertos lenguajes OO. En el caso de Db4o, el API está disponible para Java y .Net.

Consultas db4o
QBE **SODA**
Nativas

Los principales paquetes del API de Db4o son los siguientes:

Q Isabel M. Cruz Granados (Uso educativo nc)

- **com.db4o**. Paquete principal (core) de la Base de Objetos. Las interfaces y clases más importantes que incluye son:
 - **ObjectContainer**. Es el interfaz que permite realizar las principales tareas con la base de objetos. Un **ObjectContainer** puede representar **una base de datos independiente** (stand-alone) o una **conexión a un servidor** (en línea cliente-servidor). Este interfaz proporciona métodos para almacenar **store()**, consultar **queryByExample()** y eliminar **delete()** objetos de la base de datos, así como cerrar la conexión a ésta **close()**. También permite confirmar **commit** y deshacer **rollback** transacciones.
 - **EmbeddedObjectContainer**. Es un interfaz que extiende a **ObjectContainer** y representa un **ObjectContainer** local atacando a la base de datos.
 - **Db4oEmbedded**. Es una clase que proporciona métodos estáticos para conectar con la base de datos en modo embebido.
 - **ObjectServer**. Es el interfaz que permite trabajar con una base de datos db4o en modo cliente-servidor.

- `ObjectSet`. Es un interfaz que representa el conjunto de objetos devueltos por una consulta.
- `com.db4o.query`. Paquete con funcionalidades de consulta. Proporciona interfaces que permiten establecer las condiciones y criterios de una consulta y una clase para realizar consultas mediante Native Query (Consultas nativas).
- `com.db4o.config`. Paquete con funcionalidades de configuración. Contiene interfaces y clases que nos permiten configurar y/o personalizar la base de objetos según necesidades. La configuración de la base de objetos se hace por norma general antes de abrir la sesión en la misma.
 - `EmbeddedConfiguration`. Es la interface de configuración para el uso en modo embebido.

Siempre que trabajemos con bases de objetos Db4o utilizaremos el interface `ObjectContainer`, puesto que es quien representará a la base de objetos, sea embebida o no.



Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

Un `ObjectContainer` es un interfaz que proporciona, entre otros, los métodos `store()`, `queryByExample()`, `close()` y `delete()`.

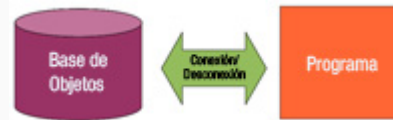
☐ Verdadero ☐ Falso

Verdadero

Es verdadero, es el interfaz que permite realizar las principales tareas con la base de datos.

4.1.- Apertura y cierre de conexiones.

En general, la conexión de una aplicación Java con una base de objetos se podrá realizar vía:



Q Isabel M. Cruz Granados (Uso educativo no)

- JDBC.
- El API proporcionado por el propio gestor de objetos.

En el caso de Db4o, el paquete `com.db4o` proporciona las clases e interfaces que permiten abrir conexiones a una base de objetos db4o, así como el cierre de la misma. Estas son:

- **Abrir conexión.** Podemos utilizar las siguientes clases:
 - `Db4oEmbedded`. Es una clase que hereda de `java.lang.Object` y proporciona métodos estáticos como `openFile()` para abrir una instancia de la base de datos en **modo embebido**. En modo embebido tiene la limitación de que sólo se puede utilizar en la base de datos una conexión.
 - `ObjectServer`. Es un interfaz que permite trabajar con una base de datos db4o en **modo cliente-servidor**. Una vez abierta la base de datos como servidor mediante `Db4o.openServer()`, el método `openClient()` del interfaz `ObjectServer` permitirá **abrir conexiones cliente directamente en memoria o bien mediante TCP/IP**.
- **Cerrar conexión.** Para ello utilizaremos el método `close()` de la interfaz `ObjectContainer`.

El **esquema de trabajo para operar con la base de objetos** será el siguiente:

- Declarar un `ObjectContainer`.
- Abrir una conexión a la base de objetos (`openFile()`). Si al abrir la BD00 esta no existe, se creará.
- Realizar operaciones con la base de objetos como consultas, borrados y modificaciones (en un bloque `try{ } catch (){ }`).
- Cierre o desconexión de la base de objetos (`close()`).

En el siguiente recurso didáctico tienes un ejemplo de apertura, creación y cierre a una base de objetos en modo embebido. En la BD00 creada se almacenan 4 objetos mediante el método `store()`.

 [Código del proyecto AD04_DB4o \(paquete crear BD\)](#) (17 KB)



Autoevaluación

Señala la opción correcta. Para cerrar una conexión a una base de datos db4o se utiliza el método:

- ☐ `closedb()`.
- ☐ `close()` de la clase `Db4oEmbedded`.
- ☐ `disconnect()` del interfaz `ObjectContainer`.
- ☐ `close()` del interfaz `ObjectContainer`.

No es correcto, prueba de nuevo.

No es la respuesta correcta, deberías haber leído mejor.

No, ese método no existe.

Muy bien, vamos por buen camino.

Solución

1. Incorrecto (#answer-28_63)
2. Incorrecto (#answer-28_108)
3. Incorrecto (#answer-28_111)
4. Opción correcta (#answer-28_114)

4.2.- Consultas a la base de objetos.

A una BDOO se podrán realizar consultas mediante:

- Un lenguaje de consultas como OQL, si el gestor está basado en el estandar ODMG e incluye sentencias del tipo SQL.
- El API proporcionado por el propio sistema gestor de bases de datos orientadas a objetos.

Los tres sistemas de consulta que proporciona Db4o basados en el API del propio gestor, son los siguientes:

- **Consultas por ejemplo. Query By Example (QBE).** Es la forma más sencilla y básica de realizar consultas, pero tienen bastantes limitaciones.
- **Consultas nativas. Native Queries (NQ).** Es la interface principal de consultas de la base de objetos. Permiten realizar un filtro contra todas las instancias de la base de objetos.
- **Consultas SODA. Simple Object Data Access (SODA).** Permite generar consultas dinámicas. Es más potente que las anteriores y más rápida puesto que las anteriores (QBE y NQ) tienen que ser traducidas a SODA para ejecutarse.



Debes conocer

Puedes estudiar el código de clase *Main.java* del *paquete consultas* del proyecto de ejemplo que aparece en el punto 4.1, donde tienes el código comentado con ejemplos de los tres tipos de consultas explicadas en los puntos anteriores.



Autoevaluación

Señala la opción correcta. Las consultas SODA en db4o:

- ☐ Son muy limitadas y no permiten el uso de restricciones.
- ☐ Necesitan de objetos **Query** para formularlas y en ellas se pueden indicar varias restricciones o **constraints**.
- ☐ No necesitan ningún API especial.
- ☐ Son más lentas que las consultas nativas.

No es correcto, prueba de nuevo.

Muy bien, vamos por buen camino.

No es cierto, precisamente utilizan la API SODA.

No, al contrario, son las más rápidas.

Solución

1. Incorrecto (#answer-30_63)
2. Opción correcta (#answer-30_120)
3. Incorrecto (#answer-30_123)
4. Incorrecto (#answer-30_126)



Para saber más

En el siguiente enlace, dispones de información sobre los sistemas de consulta del sistema gestor de objetos Neodatis.

[Texto Consultas con el gestor Neodatis](#) 

4.3.- Actualización de objetos simples.

Recuerda que un **objeto simple** es un objeto que no contiene a otros objetos, como por ejemplo los objetos de la clase `ponente`. Un segmento de la definición de esta clase, es la siguiente:

```
//Clase que implementa la entidad ponente. Objeto Simple
public class ponente {
    private String nif;
    private String nombre;
    private String email;
    private float sueldo;

    //Constructores
    public ponente() {}
    public ponente(String n, String a, String e) {}
    public ponente(String n, String a, String e, float s) {}
    //Método para obtener y guardar los datos de un ponente
    public void actualizar(String n, String a, String e, float s) {}
    public String getNif() {}
}
```

Q Isabel M. Cruz Granados (Uso educativo nc)

Para **consultar objetos simples** se pueden utilizar cualquiera de los tres sistemas de consulta proporcionados por db4o, tal y como has podido ver en el apartado anterior.

Para **modificar objetos** almacenados debes seguir los siguientes pasos:

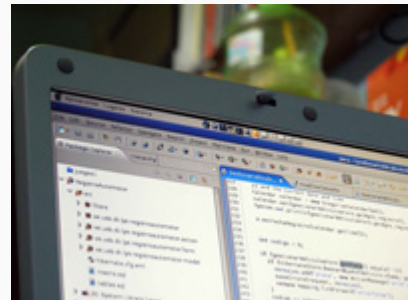
- Cambiar los valores del objeto con los nuevos valores.
- Almacenar de nuevo el objeto con el método `store()` de la interfaz `ObjectContainer`.

Por ejemplo, el siguiente método permitirá modificar objetos ponente, en concreto actualizar el e-mail de ponentes por nif de ponente:

```
//Método que actualiza el e-mail de un ponente dado su nif de paso como parámetro
// y devuelve el nif de los ponentes que fueron actualizados
public static void actualizarEmail(ObjectContainer oc, String nif, String email) {
    //Se consulta a la base de datos por el nif de los ponentes
    Iterator it = oc.queryByExample(ponente.class, nif, email, true);
    while(it.hasNext()) {
        ponente p = (ponente) it.next();
        p.setEmail(email);
        oc.store(p);
    }
}
```

Q Isabel M. Cruz Granados (Uso educativo nc)

Para eliminar objetos almacenados utilizaremos el método `delete()` de la interface `ObjectContainer`. Por ejemplo, el siguiente método elimina un objeto ponente por su nif:



Q RO __Garcia (CC BY)

```
// Método que elimina de la base de objetos [con delete()] el ponente cuyo
//nif se pasa como parámetro
public static void borrarPonentePorNif(ObjectContainer db, String nif) {
    //se consulta a la base de objetos por el ponente del nif indicado
    ObjectSet res = db.queryByExample(new ponente(nif, null, null, 0));
    ponente p = (ponente) res.next(); //se obtiene el objeto consultado en p
    db.delete(p); //se elimina el objeto poenente de la base de objetos
}
```

Q Isabel M. Cruz Granados (Uso educativo nc)

Db4o necesita conocer previamente un objeto para poder actualizarlo. Esto significa que para poder ser actualizados los objetos, éstos deben de haber sido insertados o recuperados en la misma sesión; en otro caso se añadirá otro objeto en vez de actualizarse.



Debes conocer

Puedes estudiar el código de clase *Main.java* del *paquete actualizar* del proyecto de ejemplo que aparece en el punto 4.1, donde tienes el código comentado con ejemplos de distintas operaciones que permiten la actualización de los datos.



Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

Para actualizar un objeto almacenado, db4o necesita conocerlo previamente.

☐ Verdadero ☐ Falso

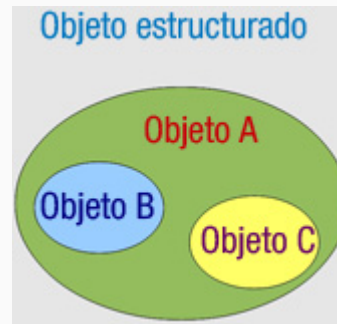
Verdadero

Es verdadero, pues si no es un objeto insertado o recuperado en la misma sesión, db4o creará un nuevo objeto.

4.4.- Actualización de objetos estructurados.

Los **objetos estructurados** son objetos que contienen a su vez a otros objetos (objetos hijo u objetos miembro).

En el caso de objetos estructurados se habla de diferentes **niveles de profundidad del objeto**. El nivel más alto, nivel 1, será el que corresponde a la definición del objeto estructurado (objeto padre), el siguiente nivel, nivel 2, corresponderá a la definición del objeto hijo y así sucesivamente podrá haber un nivel 3, 4... dependiendo de que los objetos hijos a su vez incluyan en su definición a otro u otros objetos miembro.



Q Isabel M. Cruz Granados (Uso educativo nc)

En el siguiente ejemplo, definimos la clase `charla` (objeto estructurado padre) que incorpora a un objeto `ponente` (objeto miembro). El nivel más alto de profundidad o nivel 1 es el que corresponde a la definición de `charla` y el nivel 2 corresponderá a la definición del objeto `ponente`.

¿Cómo se almacenan, consultan y actualizan los objetos estructurados en Db4o?

- Los objetos estructurados se almacenan asignando valores con `set()` y después persistiendo el objeto con `store()`. Al almacenar un objeto estructurado del nivel más alto, se almacenarán de forma implícita todos los objetos hijo.
- Las consultas se realizan por cualquiera de los sistemas soportados por el gestor y se podrá ir descendiendo por los diferentes niveles de profundidad.
- La eliminación o borrado de un objeto estructurado se realiza mediante el método `delete()`. Por defecto, no se eliminarán los objetos miembro. Para eliminar objetos estructurados en cascada o de forma recursiva, eliminando los objetos miembro, habrá que configurar de modo apropiado la base de objetos antes de abrirla, mediante el paquete `com.db4o.config`. En el caso de modo embebido, se hará mediante la interface `EmbeddedConfiguration`. En la nueva configuración se debe indicar `cascadeOnDelete(true)`.
- La modificación se realizará actualizando los nuevos valores mediante el método `set()`. Por defecto las modificaciones solo afectan al nivel más alto. Para actualizar de forma recursiva todos los objeto miembro habrá que indicar en la configuración `cascadeOnUpdate(true)`.

En este ejemplo se configura la base de datos con la opción "borrado en cascada" de `Charla`, de forma que al borrar un objeto de tipo `Charla`, eliminaremos también los `ponentes` asociados a ella.

```
//Se indica la nueva configuración de conexión, con borrado en cascada
EmbeddedConfiguration config = Db4oEmbedded.newConfiguration();
config.common().objectClass(Charla.class).cascadeOnDelete(true);
//Se abre la conexión a la base de objetos congresos.db4o
ObjectContainer db = Db4oEmbedded.openFile(config, "congresos.db4o");
```

En el siguiente ejemplo se observa como se realiza un borrado de los objetos Charla de la BD tras recuperar primeramente aquellas charlas con un título determinado. Los ponentes asociados a dichas charlas no se eliminarán por defecto, salvo que se haya indicado borrado en cascada en la configuración, como aparece indicado en el ejemplo anterior.

```
public static void borrarCharlaporTitulo(ObjectContainer db, String tit) {  
    Query query = db.query(); //declaración de un objeto query().  
    query.constrain(Charla.class); //establece la clase a la que se aplicará la restricción  
    query.descend("titulo").constrain(tit); //establece la restricción de búsqueda  
    ObjectSet resul = query.execute(); //ejecuta consulta con restricción búsqueda  
    while (resul.hasNext()) { //bucle que recupera los objetos Charla y elimina de la BDOO  
        Charla c = (Charla) resul.next();  
        System.out.println("Eliminando: " + c);  
        db.delete(c);  
    }  
}
```



Debes conocer

Puedes estudiar el código de clase *Main.java* del proyecto de ejemplo cuyo enlace aparece en el punto 4.1 donde tenemos los paquetes:

- **estructurado**: donde aparecen métodos de actualización sobre objetos relacionados entre las clases *Charla* y *Ponente*.
- **cascada**: con un método llamado *borrarCharlaporTitulo*, donde se puede ver un ejemplo de borrado en cascada.



Citas para pensar

"No basta saber, se debe también aplicar. No es suficiente querer, se debe también hacer".

Johann W. Goethe



Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

La actualización de un objeto estructurado supone la actualización de todos sus objetos hijo.

☐ Verdadero ☐ Falso

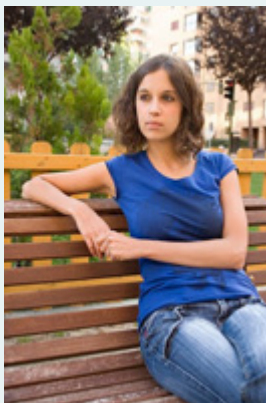
Falso

Es falso, para actualizar los objetos hijo al actualizar el objeto padre habrá que indicarlo expresamente, por ejemplo en db4o sería mediante `cascadeOnUpdate(true)`.

5.- El lenguaje de consulta de objetos OQL.



Caso práctico



Q Ministerio de
Educación (Uso educativo
nc)

Esta tarde, **Ana** ha decidido darse un respiro y dar una vuelta por el parque. Necesita descansar. Esta última semana ha sido muy intensa, —que si pruebo esta funcionalidad de Db4o, que si pruebo esta otra, ¿qué pasa con las eliminaciones en cascada? ¿Por qué no me funciona este ejemplo? —y además, ha tenido que repasar el tipo colección de Java, que lo tenía algo olvidado.

Realmente está muy entusiasmada y así se lo ha dicho a sus compañeros de clase. Mañana, empieza a repasar con **Juan** las posibilidades del lenguaje de consultas OQL. Eso ya no le preocupa tanto, **Ana** sabe que es muy parecido al SQL, y en este campo se defiende muy bien. De hecho, esta última semana, ha tenido que programar diferentes consultas para una aplicación de bases de datos de otro cliente de la empresa BK.

OQL (Object Query Language) es el lenguaje de consulta de objetos propuesto en el estándar ODMG.

Las siguientes son algunas de las **características más relevantes de OQL:**

- Es un **lenguaje declarativo**  del tipo de SQL que permite realizar consultas de modo eficiente sobre bases de datos orientadas a objetos.
- Su **sintaxis es similar a la de SQL**, proporcionando un superconjunto de la sintaxis de la sentencia **SELECT**, con algunas características añadidas para los conceptos ODMG, como la identidad del objeto, los objetos complejos, las operaciones, la herencia, el polimorfismo y las relaciones.
- **No posee primitivas para modificar el estado de los objetos** ya que las modificaciones se pueden realizar mediante los métodos que estos poseen.
- Puede ser usado como un **lenguaje autónomo o incrustado** dentro de otros lenguajes como C++, Smalltalk y Java.
- Una **consulta OQL incrustada** en uno de estos lenguajes de programación puede devolver objetos que coincidan con el sistema de tipos de ese lenguaje.
- Desde OQL se pueden invocar operaciones escritas en estos lenguajes.



Q Abraham Pérez Pérez (
CC BY-NC-SA)

■ Permite **acceso tanto asociativo como navegacional**:

- Una **consulta asociativa** devuelve una colección de objetos.
- Una **consulta navegacional** accede a objetos individuales y las interrelaciones entre objetos sirven para navegar entre objetos.



Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

El lenguaje OQL incluye un rico repertorio de sentencias para modificar el estado de los objetos.

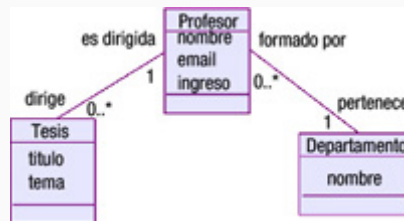
☐ Verdadero ☐ Falso

Falso

Es falso, ya que lo normal es utilizar para ello los propios métodos del objeto.

5.1.- Sintaxis, expresiones y operadores.

La sintaxis básica y resumida de una sentencia **SELECT** del OQL estándar es la siguiente:



Q Isabel M. Cruz Granados (Uso educativo nc)

```
SELECT [DISTINCT] <expresión, ...>
FROM <lista from>
[WHERE <condición> ]
[ORDER BY <expresión>]
```

Por ejemplo, suponiendo el esquema de base de objetos que puedes ver en la figura ampliable de la derecha, la siguiente sentencia select recupera de la base de objetos los atributos nombre y el correo de objetos tipo profesor cuyo año de ingreso es anterior al 1990, y ordenados alfabéticamente por nombre:

```
SELECT p.nombre, p.email FROM p in Profesor WHERE p.ingreso <= 1990 ORDER BY p.
```

Las siguientes, son algunas **consideraciones a tener en cuenta**:

- En las consultas se necesita un punto de entrada, que suele ser el nombre de una clase.
- El resultado de una consulta es una colección que puede ser tipo **bag** (si hay valores repetidos) o tipo **set** (no hay valores repetidos). En este último caso habrá que especificar **SELECT DISTINCT**.
- En general, una consulta OQL puede devolver un resultado con una estructura compleja especificada en la misma consulta utilizando **struct**.
- Una vez que se establece un punto de entrada, se pueden utilizar expresiones de caminos para especificar un camino a atributos y objetos relacionados. Una expresión de camino empieza normalmente con un nombre de objeto persistente o una variable iterador, seguida de ninguno o varios nombres de relaciones o de atributos conectados mediante un punto.
- Es posible crear objetos mutables (no literales) formados por el resultado de una consulta.

Además, en una consulta OQL se pueden utilizar, entre otros, los siguientes

operadores y expresiones:

- Operadores de acceso: `"." / "->"` aplicados a un atributo, una expresión o una relación.
`FIRST / LAST` (primero / último elemento de una lista o un vector).
- Operadores aritméticos: `+, -, *, /, -` (unario), `MOD, ABS` para formar expresiones aritméticas.
- Operadores relacionales: `>, <, >=, <=, <>, =` que permiten comparaciones y construir expresiones lógicas.
- Operadores lógicos: `NOT, AND, OR` que permiten enlazar otras expresiones.

A continuación te indicamos de manera resumida, **algunas otras características**, del estándar OQL:

- Definición de vistas, es decir, es posible dar nombre a una consulta y utilizarlo en otras consultas.
- Extracción de elementos sencillos de colecciones `set, bag`, o `list`.
- Operadores de colecciones como funciones de agregación (`MAX()`, `MIN()`, `COUNT()`, `SUM()` y `AVG()`) y cuantificadores (`FOR ALL, EXISTS`).
- Realización de agrupaciones mediante `GROUP BY` y filtro de los grupos mediante `HAVING`.
- Combinación de consultas mediante `JOINS`
- Unión, intersección y resta de colecciones mediante los operadores `UNION`, `INTERSEC` y `EXCEPT`.

En los siguientes apartados nos centraremos en el OQL concreto de un SGBD00 y que por supuesto, incorpora sus propias particularidades y diferencias respecto a la propuesta OQL de ODMG.



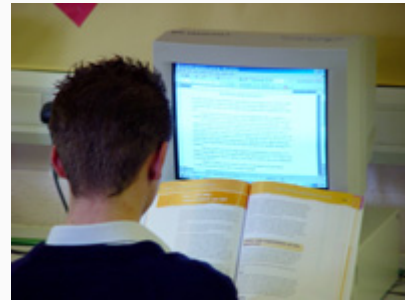
Para saber más

En el siguiente enlace puedes consultar una extensa guía de referencia y con ejemplos, en inglés, del lenguaje OQL propuesto por el ODMG.

[Guía de referencia del lenguaje OQL](#) 

5.2.- Matisse, un gestor de objetos que incorpora OQL.

Para practicar y trabajar con OQL utilizaremos Matisse, un gestor orientado a objetos que incorpora características del estándar ODMG, como los lenguajes ODL y OQL, y que tiene soporte para Java. Aunque Matisse llama SQL a su lenguaje de consultas, nosotros nos referiremos a él como OQL.



Q Paul Bangs (CC BY-NC-SA)

El propio gestor proporciona el driver `matisse.jar` para interactuar con aplicaciones escritas en Java.

Dentro de los **paquetes del API** destacamos:

- `com.matisse`. Proporciona las clases e interfaces básicos para trabajar con Java y una base de datos de objetos Matisse.
- `MtDatabase`. Clase que proporciona todos los métodos para realizar las conexiones y transacciones en la base de objetos.
- `com.matisse.sql`. Proporciona clases que permiten interactuar con la base de objetos vía JDBC.

Todas las interacciones entre una aplicación Java y la base de objetos Matisse se realizan en el contexto de una transacción (implícita o explícita).

En el siguiente recurso didáctico encontrarás un video-tutorial con los pasos a seguir para la descarga del software Matisse y realizar su instalación e integración en NetBeans a partir de del driver `matisse.jar`.

<https://www.youtube.com/embed/ACR9f3l-1Ck>



Para saber más

Desde el siguiente enlace podrás descargar numerosos manuales y abundante documentación sobre Matisse.



Documentación, manuales y guías de Matisse



5.3.- Ejecución de Sentencias OQL.

Por ejemplo, para recuperar el valor de todos los atributos de los objetos tipo Profesor, escribiríamos la siguiente sentencia OQL:

La sintaxis básica del OQL de Matisse es una sentencia `SELECT` de la forma:
`SELECT... FROM... WHERE...;`



```
SELECT * FROM Profesor;
```

Q Luana Fischer Ferreira (
CC BY-NC-SA)

Y para recuperar del atributo nombre de los objetos tipo Profesor cuyo año de ingreso es anterior al 1990, escribiríamos la siguiente sentencia:

```
SELECT nombre FROM Profesor WHERE ingreso <= 1990;
```

Las siguientes son algunas consideraciones y ejemplos sobre las consultas con `SELECT`:

- Toda sentencia `SELECT` finaliza en punto y coma.

Además de la cláusula `WHERE`, que permite establecer un criterio de selección se pueden utilizar las cláusulas de agrupamiento `GROUP BY`, y de ordenación `ORDER BY`, entre otras. También se pueden asignar alias mediante `AS`, realizar búsquedas por patrones de caracteres con `LIKE`.

Ejemplo: título y tema de las tesis cuyo tema contiene la palabra Objeto, ordenadas por tema.

```
SELECT titulo AS "Tesis", tema FROM Tesis  
WHERE tema LIKE '%Objeto%' ORDER BY tema;
```

- Al recuperar todos los atributos de una clase mediante `SELECT *`, la consulta retornará el OID de cada objeto recuperado, así como las interrelaciones o relaciones definidas para esa clase. El OID y la interrelación son del tipo `string` y se representan mediante un número hexadecimal. Realmente el identificador recuperado para la interrelación, hace referencia al primer objeto relacionado, incluso aunque la interrelación incluya a más de un objeto.

Se pueden hacer `JOIN` de clases, y por ejemplo esto puede permitir obtener todos los objetos relacionados con otro objeto en una consulta asociativa.

Ejemplo: nombre de cada profesor y título y tema de las tesis que dirige con `JOIN`

```
SELECT p.nombre, t.titulo, t.tema
FROM Profesor p JOIN Tesis t ON t.es_dirigida = p.OID;
```

Mediante **SELECT** se pueden realizar consultas navegacionales haciendo referencia a las interrelaciones entre objetos.

Ejemplo: nombre de cada profesor y título y tema de las tesis que dirige, navegacional

```
SELECT t.titulo AS "Tesis", tema, t.es_dirigida.nombre
AS "Profesor " FROM Tesis t;
```

- También se pueden realizar consultas navegacionales a través de la referencia de los objetos (**REF()**).

Ejemplo: tesis que dirigen los profesores con ingreso el 1990 o posterior

```
SELECT REF(p.dirige) FROM Profesor p
WHERE p.ingreso >= 1990;
```



Autoevaluación

Señala las opciones correctas. Las consultas OQL con **SELECT**:

- ☐ No permiten recuperar objetos relacionados.
- ☐ No permiten realizar consultas navegacionales.
- ☐ Permiten realizar JOINS.
- ☐ Permiten recuperar el OID de un objeto almacenado.

Mostrar retroalimentación

Solución

1. Incorrecto (#answer-53_58)
2. Incorrecto (#answer-53_154)
3. Correcto (#answer-53_161)

4. Correcto (#answer-53_163)

5.4.- Ejecución de sentencias OQL vía JDBC.

Veremos ahora cómo realizar consultas a una base de objetos Matisse mediante sentencias OQL embebidas en Java. Para ello, la conexión a la base de objetos se realizará vía JDBC. Matisse proporciona dos formas de manipular objetos vía JDBC: mediante JDB puro o mediante una mezcla de JDBC y Matisse.



Q Ministerio de Educación (Uso educativo no)

1. Para crear una conexión vía JDBC puro utilizaremos `java.SQL.*` y `com.Matisse.SQL.MtDriver`, no siendo necesario en este caso el paquete `com.Matisse`.

- La cadena de conexión será de la forma:

```
String url = "JDBC:mt://" + hostname + "/" + dbname;
```

- La conexión se realizará mediante

```
Connection jcon = DriverManager.getConnection(url);
```

2. Para crear una conexión vía JDBC y Matisse (a través de `MtDatabase`) se necesita `java.SQL.*` y `com.Matisse.MtDatabase`.

- La cadena de conexión será de la forma:

```
MtDatabase db = new MtDatabase(hostname, dbname);
```

- La conexión se realizará mediante: `db.open()` ; y

```
Connection jcon = db.getJDBCConnection();
```

Una vez realizada la conexión por uno u otro método, ya podremos ejecutar sentencias OQL vía JDBC, (tal y como si de una sentencia SQL se tratara) en la aplicación java. El esquema de consulta es el siguiente:

Desde el siguiente enlace puedes descargar el proyecto completo de consultas OQL vía JDBC.

 [Código con ejemplo de consultas OQL vía JDBC en Matisse](#)  (0,19 MB)



Citas para pensar

"Vale más saber alguna cosa de todo, que saberlo todo de una sola cosa".

Blaise Pascal



Autoevaluación

Señala la opción correcta. Para crear una conexión vía JDBC puro en Matisse:

- ☐ No es posible, siempre hay que utilizar una mezcla entre JDBC y Matisse.
- ☐ Además de `java.sql.*` se necesita `com.matisse.sql.MtDriver`.
- ☐ Se necesita `java.sql.*` y `com.matisse.MtDatabase`
- ☐ No se pueden realizar este tipo de conexiones en Matisse

No es correcto, prueba de nuevo.

Muy bien, vamos por buen camino.

Incorrecto, esto es justo para la mezcla Matisse y JDBC.

No es la respuesta correcta, sí que se pueden realizar.

Solución

1. Incorrecto (#answer-57_63)
2. Opción correcta (#answer-57_167)
3. Incorrecto (#answer-57_170)
4. Incorrecto (#answer-57_173)



Para saber más

En el siguiente enlace puedes consultar la Guía Matisse para programadores Java y ver diferentes ejemplos de consultas OQL y en modo nativo.

 [Guía oficial de Matisse para programadores Java](http://www.inf.ufsc.br/~r.mello/matisse/matisse_odl.pdf)  (http://www.inf.ufsc.br/~r.mello/matisse/matisse_odl.pdf) (0,64 MB)

6.- Características de las bases de datos objeto-relacionales.



Caso práctico



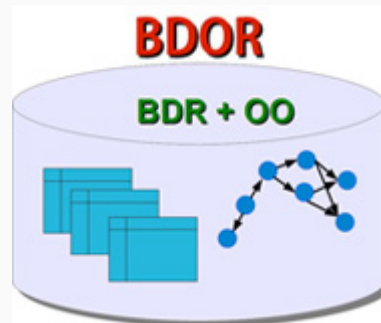
Q Ministerio de Educación (Uso educativo nc)

Hoy **Antonio** va a estar todo el día con **Juan** y **Ana**. Va a aprovechar que hoy comienzan a repasar las posibilidades de las bases de datos objeto-relacionales y como al fin y al cabo son bases de datos relacionales, quiere hacerles unas preguntillas sobre unas dudas que tiene sobre SQL, y de paso empaparse de aquellas características que tienen que ver con la orientación a objetos.

Las BDOR las podemos ver como un híbrido de las BDR y las BDOO que intenta aunar los beneficios de ambos modelos, aunque por descontado, ello suponga renunciar a algunas características de ambos.

Los objetivos que persiguen estas bases de datos son:

- Mejorar la representación de los datos mediante la orientación a objetos.
- Simplificar el acceso a datos, manteniendo el sistema relacional.



Q Isabel M. Cruz Granados. (Uso educativo nc)

En una BDOR se siguen almacenando tablas en filas y columnas, aunque la estructura de las filas no está restringida a contener escalares o valores atómicos, sino que las columnas pueden almacenar tipos estructurados (tipos compuestos como vectores, conjuntos, etc.) y las tablas pueden ser definidas en función de otras, que es lo que se denomina herencia directa.

Y eso, ¿cómo es posible?

Pues porque internamente tanto las tablas como las columnas son tratados como objetos, esto es, se realiza un mapeo objeto-relacional de manera transparente.

Como consecuencia de esto, aparecen **nuevas características**, entre las que podemos destacar las siguientes:

- **Tipos definidos por el usuario.** Se pueden crear nuevos tipos de datos definidos por el usuario, y que son compuestos o estructurados, esto es, será posible tener en una columna un atributo multivaluado (un tipo compuesto).
- **Tipos Objeto.** Posibilidad de creación de objetos como nuevo tipo de dato que permiten relaciones anidadas.

- **Reusabilidad.** Posibilidad de guardar esos tipos en el gestor de la BDOR, para reutilizarlos en tantas tablas como sea necesario
- **Creación de funciones.** Posibilidad de definir funciones y almacenarlas en el gestor. Las funciones pueden modelar el comportamiento de un tipo objeto, en este caso se llaman métodos.
- **Tablas anidadas.** Se pueden definir columnas como arrays o vectores multidimensionales, tanto de tipos básicos como de tipos estructurados, esto es, se pueden anidar tablas
- **Herencia** con subtipos y subtablas.

Estas y otras características de las bases de datos objeto-relacionales vienen recogidas en el estándar SQL 1999 .



Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

En una base de datos objeto-relacional, las columnas de una tabla pueden almacenar tipos estructurados.

☒ Verdadero ☐ Falso

Verdadero

Es verdadero, esa es una de las nuevas características de este tipo de bases de datos.

7.- Gestores de Bases de Datos Objeto-Relacionales.

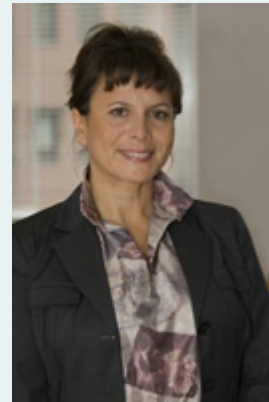


Caso práctico

Ada ha reunido a **Juan** y **Ana** esta mañana, para hablar sobre la marcha del proyecto, y decirles que **María** empezará esta semana a trabajar algunos ratos con ellos. **Ada** les dice —El cliente de la aplicación de mobiliario de diseño nos ha pedido que le diseñemos también un sitio web para su negocio, de manera que **María** se encargará de ese tema —y pregunta—, ¿os habéis decidido ya por algún sistema de bases de datos en particular?

A lo que **Juan** responde. —Esta semana vamos a valorar algunos sistemas objeto-relacionales, aunque casi estamos seguros de que para esta aplicación lo mejor es una base de objetos.

Ada sonríe y dice —Bueno, aplicaos en el tema y cuando tengáis la decisión tomada, me lo comunicáis.



Ministerio de
Educación (Uso educativo
nc)

Podemos decir que un sistema gestor de bases de datos objeto-relacional (SGBDOR) contiene dos tecnologías; la tecnología relacional y la tecnología de objetos, pero con ciertas restricciones.

A continuación te indicamos algunos ejemplos de **gestores objeto-relacionales**:

- Oracle
- PostgreSQL
- MySQL

Como base de datos objeto relacional a desarrollar se utilizará ORACLE.

El desarrollo del paradigma orientado a objetos aporta un gran cambio en el modo en que vemos los datos y los procedimientos que actúan sobre ellos. Tradicionalmente, los datos y los procedimientos se han almacenado separadamente: los datos y sus relaciones en la base de datos y los procedimientos en los programas de aplicación. La orientación a objetos, sin embargo, combina los procedimientos de una entidad con sus datos.

Esta combinación se considera como un paso adelante en la gestión de datos. Las entidades son unidades autocontenidas que se pueden reutilizar con relativa facilidad. En lugar de ligar el comportamiento de una entidad a un programa de aplicación, el comportamiento es parte de la entidad en si, por lo en cualquier lugar en el que se utilice la entidad, se comporta de un modo predecible y conocido.

El modelo orientado a objetos también soporta relaciones de muchos a muchos, siendo el primer modelo que lo permite. Aun así se debe ser muy cuidadoso

cuando se diseñan estas relaciones para evitar pérdidas de información.

Por otra parte, las bases de datos orientadas a objetos son navegacionales: el acceso a los datos es a través de las relaciones, que se almacenan con los mismos datos. Esto se considera un paso atrás. Las bases de datos orientadas a objetos no son apropiadas para realizar consultas ad hoc, al contrario que las bases de datos relacionales, aunque normalmente las soportan. La naturaleza navegacional de las bases de datos orientadas a objetos implica que las consultas deben seguir relaciones predefinidas y que no pueden insertarse nuevas relaciones “al vuelo”.

No parece que las bases de datos orientadas a objetos vayan a reemplazar a las bases de datos relacionales en todas las aplicaciones del mismo modo en que estas reemplazaron a sus predecesoras.

Los objetos han entrado en el mundo de las bases de datos de formas:

- SGBD orientados a objetos puros: son SGBD basados completamente en el modelo orientado a objetos.
- SGBD híbridos u objeto–relacionales: son SGBD relacionales que permiten almacenar objetos en sus relaciones (tablas)

El modo en que los objetos han entrado en el mundo de las bases de datos relacionales es en forma de dominios, actuando como el tipo de datos de una columna. Los objetos y los literales se categorizan en tipos. Cada tipo tiene un dominio específico compartido por todos los objetos y literales de ese tipo. Los tipos también pueden tener comportamientos. Cuando un tipo tiene comportamientos, todos los objetos de ese tipo comparten los mismos comportamientos. Hay dos implicaciones muy importantes por el hecho de utilizar una clase como un dominio: Es posible almacenar múltiples valores en una columna de una misma fila ya que un objeto suele contener múltiples valores. Sin embargo, si se utiliza una clase como dominio de una columna, en cada fila esa columna solo puede contener un objeto de la clase (se sigue manteniendo la restricción del modelo relacional de contener valores atómicos en la intersección de cada fila con cada columna). Es posible almacenar procedimientos en las relaciones porque un objeto está enlazado con el código de los procesos que sabe realizar (los métodos de su clase). Otro modo de incorporar objetos en las bases de datos relacionales es construyendo tablas de objetos, donde cada fila es un objeto. Ya que un sistema objeto–relacional es un sistema relacional que permite almacenar objetos en sus tablas, la base de datos sigue sujeta a las restricciones que se aplican a todas las bases de datos relacionales y conserva la capacidad de utilizar operaciones de concatenación (join) para implementar las relaciones “al vuelo”.

Un RDBMS (sistema gestor de bases de datos relacionales) que utiliza características orientadas a objetos tales como tipos definidos por el usuario, herencia y polimorfismo se conoce como ORDBMS (Sistema gestor de bases de datos relacionales con orientación a objetos)

7.1.- Tipos de objetos.

El modelo relacional está diseñado para representar los datos como una serie de tablas con columnas y atributos. Oracle desde la versión 8 , incorpora tecnologías orientadas a objetos. En este sentido, permite construir tipos de objetos complejos, entendidos como:

- Capacidad para definir objetos dentro de objetos.
- Cierta capacidad para encapsular o asociar métodos con dichos objetos.

Estructura de un tipo de objeto

Un tipo de objeto representa una entidad del mundo real. Encapsula datos y operaciones, por lo que en la especificación sólo se pueden declarar atributos y métodos, pero no constantes, excepciones, cursores o tipos. Al menos un atributo es requerido y los métodos son opcionales. Se compone de los siguientes elementos:

- Su nombre que sirve para identificar el tipo de los objetos.
- Sus atributos que modelan la estructura y los valores de los datos de ese tipo. Cada atributo puede ser de un tipo de datos básico o de un tipo de usuario.
- Sus métodos (procedimientos o funciones) escritos en lenguaje PL/SQL (almacenados en la BDOR), o escritos en C (almacenados externamente).

Los tipos de objetos actúan como plantillas para los objetos de cada tipo. A continuación vemos un ejemplo de cómo definir el tipo de dato Direccion_T en el lenguaje de definición de datos de Oracle, y cómo utilizar este tipo de dato para definir el tipo de dato de los objetos de la clase de Cliente_T.

```
1 CREATE TYPE direccion_t AS OBJECT (  
2   calle VARCHAR2(200),  
3   ciudad VARCHAR2(200),  
4   prov CHAR(2),  
5   codpos VARCHAR2(20));  
6  
7 CREATE TYPE cliente_t AS OBJECT (  
8   clinum NUMBER,  
9   clinomb VARCHAR2(200),  
10  direccion direccion_t, -- Uso de tipo creado anteriormente  
11  telefono VARCHAR2(20),  
12  fecha_nac DATE,  
13  MEMBER FUNCTION edad RETURN NUMBER,  
14  PRAGMA  
15  RESTRICT_REFERENCES(edad,WNDS));
```

Atributos

Como las variables, un atributo se declara mediante un nombre y un tipo. El

nombre debe ser único dentro del tipo de objeto (aunque puede reutilizarse en otros objetos) y el tipo puede ser cualquier tipo de Oracle excepto:

- LONG y LONG RAW.
- NCHAR, NCLOB y NVARCHAR2.
- MLSLABEL y ROWID.
- Los tipos específicos de PL/SQL: BINARY_INTEGER (y cualquiera de sus subtipos), BOOLEAN, PLS_INTEGER, RECORD, REF CURSOR, %TYPE y %ROWTYPE.
- Los tipos definidos en los paquetes PL/SQL.

Tampoco se puede inicializar un atributo en la declaración empleando el operador de asignación o la cláusula DEFAULT, del mismo modo que no se puede imponer la restricción NOT NULL. Sin embargo, los objetos se pueden almacenar en tablas de la base de datos en las que sí es posible imponer restricciones.

Métodos

En general, un método es un subprograma declarado en una especificación de tipo mediante la palabra clave MEMBER. El método no puede tener el mismo nombre que el tipo de objeto ni el de ninguno de sus atributos.

Muchos métodos constan de dos partes: especificación y cuerpo. La especificación consiste en el nombre del método, una lista opcional de parámetros y en el caso de funciones un tipo de retorno. El cuerpo es el código que se ejecuta para llevar a cabo una operación específica.

Para cada especificación de método en una especificación de tipo debe existir el correspondiente cuerpo del método. El PL/SQL compara la especificación del método y el cuerpo token a token, por lo que las cabeceras deben coincidir palabra a palabra.

En un tipo de objeto, los métodos pueden hacer referencia a los atributos y a los otros métodos sin cualificador. Los métodos se pueden ejecutar sobre los objetos de su mismo tipo. Si x es una variable PL/SQL que almacena objetos del tipo Cliente_T, entonces `x.getEdad()` calcula la edad del cliente almacenado en x. La definición del cuerpo de un método en PL/SQL se hace de la siguiente manera:

```
1  -- Creación del método getEdad de la clase cliente_t (Create type)
2  CREATE OR REPLACE TYPE BODY cliente_t AS
3  MEMBER FUNCTION getEdad RETURN NUMBER IS
4      a NUMBER;
5      d DATE;
6  BEGIN
7      d:= today();
8      a:= d.año - fecha_nac.año;
9      IF (d.mes < fecha_nac.mes) OR ((d.mes = fecha_nac.mes) AND (d.día < fecha_nac.día)) THEN
10         a:= a-1;
11     END IF;
```



```

12      RETURN a;
13  END;
14  END;
15

```

El parámetro SELF

Todos los métodos de un tipo de objeto aceptan como primer parámetro una instancia predefinida del mismo tipo denominada SELF. Independientemente de que se declare implícita o explícitamente, SELF es siempre el primer parámetro pasado a un método. Por ejemplo, el método transform declara SELF como un parámetro IN OUT:

```

CREATE TYPE Complex AS OBJECT (
  MEMBER FUNCTION transform ( SELF IN OUT Complex ) . . .

```

El modo de acceso por omisión de SELF, es decir, cuando no se declara explícitamente es:

- En funciones miembro el acceso de SELF es IN.
- En procedimientos, si SELF no se declara, su modo por omisión es IN OUT.

En el cuerpo de un método, SELF denota al objeto a partir del cual se invocó el método. Como muestra el siguiente ejemplo, los métodos pueden hacer referencia a los atributos de SELF sin necesidad de utilizar un cualificador:

```

1  -- Creación de una función autónoma no definida dentro de ninguna clase
2  CREATE FUNCTION gcd ( x INTEGER , y INTEGER ) RETURN INTEGER AS
3  -- Encuentra el máximo común divisor de x e y
4  ans INTEGER;
5  BEGIN
6      IF x < y THEN
7          ans := gcd ( x , y ) ;
8      ELSE
9          ans := gcd ( y , x MOD y ) ;
10     ENDIF ;
11     RETURN ans ;
12 END;
13 /
14 -- Nos creamos una clase nueva llamada Rational
15 CREATE TYPE Rational AS
16     num INTEGER ,
17     den INTEGER ,
18 MEMBER PROCEDURE normalize ,
19 . . .
20 ) ;
21 /
22 CREATE TYPE BODY Rational AS MEMBER PROCEDURE normalize IS
23     g INTEGER;

```

```

24 BEGIN
25     --Estas dos sentencias son equivalentes
26     g := gcd ( SELF . num , SELF . den ) ; -- Atributos de clase Rati
27     g := gcd ( num , den ) ;
28     num := num / g ;
29     den := den / g ;
30 END normalize ;
31 . . .
32 END;
33 /

```

Constructores

En Oracle, todos los tipos de objetos tienen asociado por defecto un método que construye nuevos objetos de ese tipo de acuerdo a la especificación del tipo. El nombre del método coincide con el nombre del tipo, y sus parámetros son los atributos del tipo. Por ejemplo las siguientes expresiones construyen dos objetos con todos sus valores.

```

1 direccion_t('Avenida Sagunto', 'Puzol', 'Valencia', 'E-23523')
2 cliente_t( 2347, 'Juan Pérez Ruíz', direccion_t('Calle Eo', 'Ond

```

Métodos de comparación

Para comparar los objetos de cierto tipo es necesario indicar a Oracle cuál es el criterio de comparación. Para ello, hay que escoger entre un método MAP u ORDER, debiéndose definir al menos uno de estos métodos por cada tipo de objeto que necesite ser comparado. La diferencia entre ambos es la siguiente:

- Un método MAP sirve para indicar cuál de los atributos del tipo se utilizará para ordenar los objetos del tipo, y por tanto se puede utilizar para comparar los objetos de ese tipo por medio de los operadores de comparación aritméticos (<, >). El PL/SQL usa esta función para evaluar expresiones booleanas como $x > y$ y para las comparaciones implícitas que requieren las cláusulas DISTINCT, GROUP BY y ORDER BY. Un tipo de objeto puede contener sólo una función de MAP, que necesariamente debe carecer de parámetros y debe devolver uno de los siguientes tipos escalares: DATE, NUMBER, VARCHAR2 y cualquiera de los tipos ANSI SQL (como CHARACTER o REAL). Por ejemplo, la siguiente declaración permite decir que los objetos del tipo cliente_t se van a comparar por su atributo clinum.

```

1 -- Creación del equals basado en el número de cliente (clinum)
2 CREATE OR REPLACE TYPE BODY cliente_t AS
3 MAP MEMBER FUNCTION ret_value RETURN NUMBER IS
4 BEGIN
5     RETURN clinum
6 END;
7 END;

```

- Un método ORDER utiliza los atributos del objeto sobre el que se ejecuta para realizar un cálculo y compararlo con otro objeto del mismo tipo que toma como argumento de entrada. Este método devolverá un valor negativo si el parámetro de entrada es mayor que el atributo, un valor positivo si ocurre lo contrario y un cero si ambos son iguales. El siguiente ejemplo define un orden para el tipo cliente_t diferente al anterior. Sólo una de estas definiciones puede ser válida en un tiempo dado.

```
1 CREATE TYPE cliente_t AS OBJECT (  
2     clinum NUMBER,  
3     clinomb VARCHAR2(200),  
4     direccion direccion_t,  
5     telefono VARCHAR2(20),  
6     fecha_nac DATE,  
7     ORDER MEMBER FUNCTION -- Definición del equals  
8     cli_ordenados (x IN clientes_t) RETURN INTEGER);  
9  
10    -- Implementación de dicho equals  
11    CREATE OR REPLACE TYPE BODY cliente_t AS  
12        ORDER MEMBER FUNCTION cli_ordenados (x IN cliente_t) RETURN  
13    BEGIN  
14        RETURN clinum - x.clinum; /*la resta de los dos números cl  
15    END;  
16 END;
```

Veamos otro ejemplo: supongamos que c1 y c2 son objetos del tipo *Customer*. Una comparación del tipo `c1 > c2` invoca al método `match` automáticamente. El método devuelve un número negativo, cero o positivo que indica que SELF es respectivamente menor, igual o mayor que el otro parámetro:

```
1 -- Creación de la clase Cliente (Customer)  
2 CREATE TYPE Customer AS OBJECT (  
3     id NUMBER,  
4     name VARCHAR2( 20 ) ,  
5     addr VARCHAR2( 30 ) ,  
6     ORDER MEMBER FUNCTION match ( c Customer ) RETURN INTEGER  
7 ) ;  
8 -- Implementación del equals por id  
9 CREATE TYPE BODY Customer AS  
10     ORDER MEMBER FUNCTION match ( c Customer ) RETURN INTEGER IS  
11 BEGIN  
12     IF id < c .id THEN  
13         RETURN -1;  
14     --Cualquier numero negativo .  
15     ELSEIF id > c . id THEN  
16         RETURN 1 ;  
17     --cualquier numero positivo  
18     ELSE  
19         RETURN 0 ;  
20     END IF ;  
21 END;
```

Un tipo de objeto puede contener un único método ORDER, que es una función que devuelve un resultado numérico. Es importante tener en cuenta los siguientes puntos:

- Un método MAP proyecta el valor de los objetos en valores escalares (que son más fáciles de comparar). Un método ORDER simplemente compara el valor de un objeto con otro.
- Se puede declarar un método MAP o un método ORDER, pero no ambos.
- Si se declara uno de los dos métodos, es posible comparar objetos en SQL o en un procedimiento.
- Cuando es necesario ordenar un número grande de objetos es mejor utilizar un método MAP (ya que una llamada por objeto proporciona una proyección escalar que es más fácil de ordenar). Un método ORDER es menos eficiente: debe invocarse repetidamente ya que compara sólo dos objetos cada vez

Si un tipo de objeto no tiene definido ninguno de estos métodos, Oracle es incapaz de deducir cuándo un objeto es mayor o menor que otro.

Sin embargo, sí puede determinar cuándo dos objetos del mismo tipo son iguales. Para ello, el sistema compara el valor de los atributos de los objetos uno a uno:

- Si todos los atributos son no nulos e iguales, Oracle indica que ambos objetos son iguales.
- Si alguno de los atributos no nulos es distinto en los dos objetos, entonces Oracle dice que son diferentes.
- En otro caso, Oracle dice que no puede comparar ambos objetos.

Declaración e inicialización de objetos

Una vez que se ha definido un tipo de objeto y se ha instalado en el esquema de la base de datos, es posible usarlo en cualquier bloque PL/SQL. Las instancias de los objetos se crean en tiempo de ejecución. Estos objetos siguen las reglas normales de ámbito y de instanciación. En un bloque o subprograma, los objetos locales son instanciados cuando se entra en el bloque o subprograma y dejan de existir cuando se sale. En un paquete, los objetos se instancian cuando se referencia por primera vez al paquete y dejan de existir cuando finaliza la sesión.

Declaración de objetos

Los tipos de objetos se declaran del mismo modo que cualquier tipo interno. Por ejemplo, en el bloque que sigue se declara un objeto r de tipo Racional y se invoca al constructor para asignar su valor.

La llamada asigna los valores 6 y 8 a los atributos num y den

respectivamente:

```
1 DECLARE
2     r Racional ;
3 BEGIN
4     r: = Racional (6, 8); -- Creamos un objeto
5     DBMS_OUTPUT.PUT_LINE(r.num) ; -- Mostramos por pantalla el valor 6
6 END;
```

También es posible declarar objetos como parámetros formales de funciones y procedimientos, de modo que es posible pasar objetos a los subprogramas almacenados y de un subprograma a otro. En el siguiente ejemplo, se emplea un objeto de tipo Account para especificar el tipo de dato de un parámetro formal:

```
1 DECLARE
2     . . .
3 PROCEDURE open_act ( new_acct IN OUT Account ) IS . . .
```

y en el siguiente ejemplo se declara una función que devuelve un objeto de tipo Account:

```
1 DECLARE
2     . . .
3 FUNCTION get_act( act_id IN INTEGER ) RETURN Account IS . . .
```

Inicialización de objetos

Hasta que se inicializa un objeto, invocando al constructor para ese tipo de objeto, el objeto se dice que es Atómicamente nulo. Esto es, el objeto es nulo, no sólo sus atributos.

Un objeto nulo siempre es diferente a cualquier otro objeto. De hecho, la comparación de un objeto nulo con otro objeto siempre resulta NULL. Del mismo modo, si se asigna un objeto con otro objeto atómicamente nulo, el primero se convierte a su vez en un objeto atómicamente nulo (y para poder utilizarlo debe ser reinicializado).

En resumen, si asignamos el no-valor NULL a un objeto, éste se convierte en atómicamente nulo, como se ilustra en el siguiente ejemplo:

```
1 DECLARE
2     r Racional;
3 BEGIN
4     r Racional : = Racional (1, 2) ; -- Nos creamos un objeto de tipo
5     r := NULL; -- Le asignamos NULL al objeto
6     IF r IS NULL THEN . . .-- la condición devuelve TRUE
```

Una buena práctica de programación consiste en inicializar los objetos en su declaración, como se muestra en el siguiente ejemplo:

```
1 DECLARE
2     r Racional := Racional (2 , 3) ; -- Creación e invocación al constructor
```

Objetos sin inicializar en PL/SQL

PL/SQL se comporta del siguiente modo cuando accede a objetos sin inicializar:

- Los atributos de un objeto no inicializado se evalúan en cualquier expresión como NULL.
- Intentar asignar valores a los atributos de un objeto sin inicializar provoca la excepción predefinida ACCESS_INTO_NULL.
- La operación de comparación IS NULL siempre produce TRUE cuando se aplica a un objeto no inicializado o a cualquiera de sus atributos.

Existe por tanto, una sutil diferencia entre objetos nulos y objetos con atributos nulos. El siguiente ejemplo intenta ilustrar esa diferencia:

```
1 DECLARE
2     r Racional ; -- r vale NULL inicialmente
3 BEGIN
4     IF r IS NULL THEN . . . --TRUE
5         IF r.num IS NULL THEN . . . --TRUE
6             r := Racional (NULL , NULL) ; -- Inicializar num=den=NULL
7             r.num := 4 ; -- Modificamos el atributo num de r que deja
8             r := NULL ; -- El objeto r es ahora NULL
9             r.num := 4 ; -- Provoca la excepcion ACCESS_INTO_NULL. Ya
10 EXCEPTION
11     WHEN ACCESS_INTO_NULL THEN
12         . . .
13 END;
14 /
```

La invocación de los métodos de un objeto no inicializado está permitida, pero en este caso: SELF toma el valor NULL. Cuando los atributos de un objeto no inicializado se pasan como parámetros IN, se evalúan como NULL. Cuando los atributos de un objeto no inicializado se pasan como parámetros OUT o IN OUT, se produce una excepción si se intenta asignarles un valor.

Acceso a los atributos

Para acceder o cambiar los valores de un atributo se emplea la notación punto ('.'). El siguiente ejemplo ilustra esa notación:

```
1 DECLARE
```

```

2      r Racional := Racional (NULL , NULL) ; -- Instanciamos un objeto
3      numerador INTEGER;
4      denominador INTEGER;
5  BEGIN
6      . . .
7      -- Movemos datos entre variables y atributos
8      denominador := r.den;
9      r.num := numerador;

```

Los nombres de los atributos pueden encadenarse, lo que permite acceder a los atributos de un tipo de objeto anidado. Por ejemplo, supongamos que definimos los tipos de objeto *Address* y *Student* como sigue:

```

1  CREATE TYPE Address AS OBJECT (
2      street VARCHAR2(30),
3      city VARCHAR2(20),
4      state CHAR(2),
5      zip_code VARCHAR2(5)
6  ) ;
7  /
8  CREATE TYPE Student AS OBJECT (
9      name VARCHAR2(20) , -- tipo escalar básico
10     home_address Address, -- Atributo de tipo Object
11     phone_number VARCHAR2(10),
12     status VARCHAR2(10) ,
13     advisor_name VARCHAR2(20),
14     . . .
15 ) ;
16 /

```

Observe que *zip_code* es un atributo de tipo *Address* y que *Address* es el tipo de dato del atributo *home_address* del tipo de objeto *Student*. Si *s* es un objeto *Student*, para acceder al valor de su *zip_code* se emplea la siguiente notación:

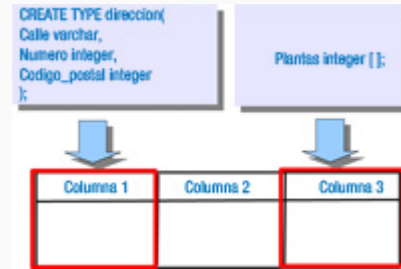
```

1  s.home_address.zip_code

```

7.2.- Tablas de objetos

Una vez definidos los tipos, éstos pueden utilizarse para definir nuevos tipos, tablas que almacenen objetos de esos tipos, o para definir el tipo de los atributos de una tabla. Una tabla de objetos es una clase especial de tabla que almacena un objeto en cada fila y que facilita el acceso a los atributos de esos objetos como si fueran columnas de la tabla. Por ejemplo, se puede definir una tabla para almacenar los clientes de este año y otra para almacenar los de años anteriores de la siguiente manera:



Q Isabel Cruz Granados (Uso educativo nc)

```
1  -- Creación de una tabla basada en la clase cliente_t
2  CREATE TABLE clientes_año_tab OF cliente_t (
3      clinum PRIMARY KEY
4  );
5
6  -- Creación de una tabla con un atributo de tipo objeto cliente_t
7  CREATE TABLE clientes_antiguos_tab (
8      año NUMBER,
9      cliente cliente_t
10 );
```

La diferencia entre la primera y la segunda tabla es que la primera almacena objetos con su propia identidad (OID) y la segunda no es una tabla de objetos, sino una tabla con una columna con un tipo de datos de objeto. Es decir, la segunda tabla tiene una columna con un tipo de datos complejo pero sin identidad de objeto. Además de esto, Oracle permite considerar una tabla de objetos desde dos puntos de vista:

- Como una tabla con una sola columna cuyo tipo es el de un tipo de objetos.
- Como una tabla que tiene tantas columnas como atributos los objetos que almacena.

Por ejemplo, se puede ejecutar una de las dos instrucciones siguientes. En la primera instrucción, la tabla `clientes_año_tab` se considera como una tabla con varias columnas cuyos valores son los especificados. En el segundo caso, se la considera como con una tabla de objetos que en cada fila almacena un objeto. En esta instrucción la cláusula `VALUE` permite visualizar el valor de un objeto.

```
1  INSERT INTO clientes_año_tab VALUES( 2347, 'Juan Pérez Ruíz', direccion_
2
3  SELECT VALUE(c)
4  FROM clientes_año_tab c
5  WHERE c.clinomb = 'Juan Pérez Ruíz';
```

Las reglas de integridad, de clave primaria, y el resto de propiedades que se

definan sobre una tabla, sólo afectan a los objetos de esa tabla, es decir no se refieren a todos los objetos del tipo asignado a la tabla.



Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

En la orientación a objetos relacionales las columnas pueden albergar datos simples y/o complejos

☒ Verdadero ☐ Falso

Verdadero

Pueden ser ambos tipos

7.3.- Referencias entre objetos

Los identificadores únicos asignados por Oracle a los objetos que se almacenan en una tabla, permiten que éstos puedan ser referenciados desde los atributos de otros objetos o desde las columnas de tablas. El tipo de datos proporcionado por Oracle para soportar esta facilidad se denomina REF. Un atributo de tipo REF almacena una referencia a un objeto del tipo definido, e implementa una relación de asociación entre los dos tipos de objetos. Estas referencias se pueden utilizar para acceder a los objetos referenciados y para modificarlos; sin embargo, no es posible operar sobre ellas directamente. Para asignar o actualizar una referencia se debe utilizar siempre REF o NULL.

Cuando se define una columna de un tipo a REF, es posible restringir su dominio a los objetos que se almacenen en cierta tabla. Si la referencia no se asocia a una tabla sino que sólo se restringe a un tipo de objeto, se podrá actualizar a una referencia a un objeto del tipo adecuado con independencia de la tabla donde se almacene. En este caso su almacenamiento requerirá más espacio y su acceso será menos eficiente. El siguiente ejemplo define un atributo de tipo REF y restringe su dominio a los objetos de cierta tabla.

```
1 CREATE TABLE clientes_tab OF cliente_t;  
2  
3 CREATE TYPE ordenes_t AS OBJECT (  
4     ordnum NUMBER,  
5     cliente REF clientes_t,  
6     fechpedido DATE,  
7     direcentrega direccion_t);  
8  
9 CREATE TABLE ordenes_tab OF ordenes_t (  
10     PRIMARY KEY (ordnum)  
11 );
```

No es posible navegar a través de referencias en procedimientos SQL. Para esto es necesario utilizar el operador Deref (abreviatura del término inglés dereference : derreferenciar un puntero es obtener el valor al cual apunta). Deref toma como argumento una referencia a un objeto y devuelve el valor de dicho objeto. Si la referencia está colgada, Deref devuelve el valor NULL.

En el ejemplo que sigue se derreferencia una referencia a un objeto *Person* de la tabla *DUAL2*. En estas circunstancias, no es necesario especificar una tabla de objetos ni un criterio de búsqueda ya que cada objeto almacenado en una tabla de objetos cuenta con un identificador de objeto único e inmutable que es parte de cada referencia a un objeto.

```
1 DECLARE  
2     p1 Person;  
3     p_ref REF Person;  
4     name VARCHAR2(15);  
5  
6 BEGIN
```

```

7   ...
8   -- Supongamos que p_ref contiene una referencia valida a un objeto almac
9
10  -- Podemos hacer
11  SELECT Deref( p_ref) INTO p1 FROM DUAL;
12  name := p1.last_name ;

```

Es posible utilizar el operador DREF en sentencias SQL sucesivas para derr

```

1  CREATE TYPE PersonRef AS OBJECT (pref REF Person) ;
2  /
3  DECLARE
4      name VARCHAR2(15);
5      pr_ref REF Pe r sonRe f;
6      pr PersonRef;
7      p Person;
8  BEGIN
9      . . .
10     -- Supongamos que pr_ref contiene una referencia valida
11     SELECT Deref( pr_ref ) INTO pr FROM DUAL;
12     SELECT Deref( pr.p_ref ) INTO p FROM DUAL;
13     name := p.last_name ;
14     ...
15 END;
16 /

```

7.4.- Tipos de datos colección

Para poder implementar relaciones 1:N, Oracle permite definir tipos colección. Un dato de tipo colección está formado por un número indefinido de elementos, todos del mismo tipo. De esta manera, es posible almacenar en un atributo un conjunto de tuplas en forma de array (VARRAY), o en forma de tabla anidada.

Al igual que los tipo objeto, los tipo colección también tienen por defecto unas funciones constructoras de colecciones cuyo nombre coincide con el del tipo. Los argumentos de entrada de estas funciones son el conjunto de elementos que forman la colección separados por comas y entre paréntesis, y el resultado es un valor del tipo colección.

En Oracle es posible diferenciar entre un valor nulo y una colección vacía. Para construir una colección sin elementos se puede utilizar la función constructora del tipo seguida por dos paréntesis sin elementos dentro.

El tipo VARRAY

Un array es un conjunto ordenado de elementos del mismo tipo. Cada elemento tiene asociado un índice que indica su posición dentro del array. Oracle permite que los VARRAY sean de longitud variable, aunque es necesario especificar un tamaño máximo cuando se declara el tipo VARRAY. Las siguientes declaraciones crean un tipo para una lista ordenada de precios, y un valor para dicho tipo.

```
1  -- Nos definimos el atributo precios de tipo array de enteros de
2  CREATE TYPE precios AS VARRAY(10) OF NUMBER(12);
3
4  -- Esta sería la forma de utilizarlo
5  precios('35', '342', '3970');
```

Se puede utilizar el tipo VARRAY para:

- Definir el tipo de dato de una columna de una tabla relacional.
- Definir el tipo de dato de un atributo de un tipo de objeto.
- Para definir una variable PL/SQL, un parámetro o el tipo que devuelve una función.

Cuando se declara un tipo VARRAY no se produce ninguna reserva de espacio. Si el espacio que requiere lo permite, se almacena junto con el resto de columnas de su tabla, pero si es demasiado largo (más de 4000 bytes) se almacena aparte de la tabla como un BLOB.

En el siguiente ejemplo, se define un tipo de datos para almacenar una lista ordenada de teléfonos: el tipo list (ya que en el tipo set no existe orden). Este tipo se utiliza después para asignárselo a un atributo del tipo de objeto cliente_t.

```

1  -- Lista de teléfonos es un array de Strings
2  CREATE TYPE lista_tel_t AS VARRAY(10) OF VARCHAR2(20);
3
4  -- Usamos los tipos definidos en ejemplos anteriores para crear
5  CREATE TYPE cliente_t AS OBJECT (
6      clinum NUMBER,
7      clinomb VARCHAR2(200),
8      direccion direccion_t,
9      lista_tel lista_tel_t
10 );

```

La principal limitación del tipo VARRAY es que en las consultas es imposible poner condiciones sobre los elementos almacenados dentro. Desde una consulta SQL, los valores de un VARRAY solamente pueden ser accedidos y recuperados como un bloque. Es decir, no se puede acceder individualmente a los elementos de un VARRAY. Sin embargo, desde un programa PL/SQL si que es posible definir un bucle que itere sobre los elementos de un VARRAY.³ Se define el tipo objeto ordenes_t y su atributo pedido almacena una tabla anidada del tipo lineas_pedido_t.

Tablas anidadas

Una tabla anidada es un conjunto de elementos del mismo tipo sin ningún orden predefinido. Estas tablas solamente pueden tener una columna que puede ser de un tipo de datos básico de Oracle, o de un tipo de objeto definido por el usuario. En este último caso, la tabla anidada también puede ser considerada como una tabla con tantas columnas como atributos tenga el tipo de objeto.

El siguiente ejemplo declara una tabla que después será anidada en el tipo ordenes_t. Los pasos de todo el diseño son los siguientes:

1. Se define el tipo de objeto *linea_t* para las filas de la tabla anidada.

```

1  CREATE TYPE linea_t AS OBJECT (
2      linum NUMBER,
3      item VARCHAR2(30),
4      cantidad NUMBER,
5      descuento NUMBER(6,2)
6  );

```

2. Se define el tipo colección tabla *lineas_pedido_t* para después anidarla.

```

1  CREATE TYPE lineas_pedido_t AS TABLE OF linea_t ;

```

Esta definición permite utilizar el tipo colección *lineas_pedido_t* para:

- Definir el tipo de dato de una columna de una tabla relacional.
- Definir el tipo de dato de un atributo de un tipo de objetos.
- Para definir una variable PL/SQL, un parámetro, o el tipo que devuelve una función.

3. Se define el tipo objeto `ordenes_t` y su atributo `pedido` almacena una tabla anidada del tipo `lineas_pedido_t`.

```
1 CREATE TYPE ordenes_t AS OBJECT (  
2     ordnum NUMBER,  
3     cliente REF cliente_t,  
4     fechpedido DATE,  
5     fechentrega DATE,  
6     pedido lineas_pedido_t,  
7     direcentrega direccion_t  
8 ) ;
```

4. Se define la tabla de objetos `ordenes_tab` y se especifica la tabla anidada del tipo `lineas_pedido_t`.

```
1 CREATE TABLE ordenes_tab OF ordenes_t (  
2     ordnum PRIMARY KEY,  
3     SCOPE FOR (cliente) IS clientes_tab)  
4     NESTED TABLE pedido STORE AS pedidos_tab  
5 ) ;
```

Este último paso es necesario realizarlo porque la declaración de una tabla anidada no reserva ningún espacio para su almacenamiento. Lo que se hace es indicar en qué tabla (`pedidos_tab`) se deben almacenar todas las líneas de pedido que se representen en el atributo `pedido` de cualquier objeto de la tabla `ordenes_tab`. Es decir, todas las líneas de pedido de todas las ordenes se almacenan externamente a la tabla de ordenes, en otra tabla especial. Para relacionar las tuplas de una tabla anidada con la tupla a la que pertenecen, se utiliza una columna oculta que aparece en la tabla anidada por defecto. Todas las tuplas de una tabla anidada que pertenecen a la misma tupla tienen el mismo valor en esta columna (`NESTED_TABLE_ID`).

A diferencia de los `VARRAY`, los elementos de las tablas anidadas (`NESTED_TABLE`) sí pueden ser accedidos individualmente, y es posible poner condiciones de recuperación sobre ellos.

Más adelante, veremos, una forma conveniente de acceder individualmente a los elementos de una tabla anidada mediante un cursor anidado. Además, las tablas anidadas pueden estar indexadas.



Citas para pensar

"No es sabio el que sabe donde está el tesoro, sino el que trabaja y lo saca".

Francisco de Quevedo

7.5.- Acceso a datos

A continuación te presentamos algunas cuestiones relacionadas con el acceso a datos:

Alias

En una base de datos con tipos y objetos, lo más recomendable es utilizar siempre alias para los nombres de las tablas. El alias de una tabla debe ser único en el contexto de la consulta. Los alias sirven para acceder al contenido de la tabla, pero hay que saber utilizarlos adecuadamente en las tablas que almacenan objetos.

El siguiente ejemplo ilustra cómo se deben utilizar.

```
1 CREATE TYPE persona AS OBJECT (nombre VARCHAR(20));
2 CREATE TABLE ptab1 OF persona;
3 CREATE TABLE ptab2 (c1 persona);
4 CREATE TABLE ptab3 (c1 REF persona);
```

La diferencia entre las dos primeras tablas está en que la primera almacena objetos del tipo persona, mientras que la segunda tabla tiene una columna donde se almacenan valores del tipo persona. Considerando ahora las siguientes consultas, se ve cómo se puede acceder a estas tablas.

```
1 SELECT nombre FROM ptab1; Correcto
2 SELECT c1.nombre FROM ptab2; Incorrecto
3 SELECT p.c1.nombre FROM ptab2 p; Correcto
4 SELECT p.c1.nombre FROM ptab3 p; Correcto
5 SELECT p.nombre FROM ptab3 p; Incorrecto
```

En la primera consulta nombre es considerado como una de las columnas de la tabla ptab1, ya que los atributos de los objetos se consideran columnas de la tabla de objetos. Sin embargo, en la segunda consulta se requiere la utilización de un alias para indicar que nombre es el nombre de un atributo del objeto de tipo persona que se almacena en la columna c1. Para resolver este problema no es posible utilizar los nombres de las tablas directamente: ptab2.c1.nombre es incorrecto. Las consultas 4 y 5 muestran cómo acceder a los atributos de los objetos referenciados desde un atributo de la tabla ptab3.

En conclusión, para facilitar la formulación de consultas y evitar errores se recomienda utilizar alias para acceder a todas las tablas que contengan objetos con o sin identidad, y para acceder a las columnas de las tablas en general.

Inserción de referencias

La inserción de objetos con referencias implica la utilización del operador REF para poder insertar la referencia en el atributo adecuado. La siguiente sentencia inserta una orden de pedido en la tabla definida en la sección 1.1.3.

```
1 | INSERT INTO ordenes_tab
2 |   SELECT 3001, REF(C), '30-MAY-1999', NULL
3 |   FROM cliente_tab C WHERE C.clinum= 3;
4 | -- Se seleccionan los valores de los 4 atributos de la tabla
```

El acceso a un objeto desde una referencia REF requiere primero referenciar al objeto. Para realizar esta operación, Oracle proporciona el operador Deref. No obstante, utilizando la notación de punto también se consigue referenciar a un objeto de forma implícita. Observemos el siguiente ejemplo:

```
1 | CREATE TYPE persona_t AS OBJECT (
2 |     nombre VARCHAR2(30),
3 |     jefe REF persona_t
4 | );
```

Si x es una variable que representa a un objeto de tipo persona_t, entonces las dos expresiones siguientes son equivalentes:

```
1 | x.jefe.nombre
2 | y.nombre, y=Deref(x.jefe)
```

Para obtener una referencia a un objeto de una tabla de objetos, se puede aplicar el operador REF como muestra el siguiente ejemplo:

```
1 | CREATE TABLE persona_tab OF persona_t;
2 |
3 | DECLARE ref_persona REF persona_t;
4 |
5 | SELECT REF(pe) INTO ref_persona
6 | FROM persona_tab pe WHERE pe.nombre= 'Juan Pérez Ruíz';
```

Simétricamente, para recuperar un objeto desde una referencia es necesario usar Deref, como muestra el siguiente ejemplo que visualiza los datos del jefe de la persona indicada:

```
1 | SELECT Deref(pe.jefe)
2 | FROM persona_tab pe WHERE pe.nombre= 'Juan Pérez Ruíz';
```

Llamadas a métodos

Para invocar un método hay que utilizar su nombre y unos paréntesis que encierren sus argumentos de entrada. Si el método no tiene argumentos, se especifican los paréntesis aunque estén vacíos. Por ejemplo, si `tb` es una tabla con la columna `c` de tipo de objeto `t`, y `t` tiene un método `m` sin argumentos de entrada, la siguiente consulta es correcta:

```
1 | SELECT p.c.m FROM tb p;
```

Inserción en tablas anidadas

Además del constructor del tipo de colección disponible por defecto, la inserción de elementos dentro de una tabla anidada puede hacerse siguiendo estas dos etapas:

1. Crear el objeto con la tabla anidada y dejar vacío el campo que contiene las tuplas anidadas.
2. Comenzar a insertar tuplas en la columna correspondiente de la tupla seleccionada por una subconsulta.

Para ello, se tiene que utilizar la palabra clave `THE` con la siguiente sintaxis:

```
1 | INSERT INTO THE (subconsulta) (tuplas a insertar)
```

Esta técnica es especialmente útil si dentro de una tabla anidada se guardan referencias a otros objetos. El siguiente ejemplo ilustra la manera de realizar estas operaciones sobre la tabla de ordenes (`ordenes_tab`) definida anteriormente.

```
1 | INSERT INTO ordenes_tab --inserta una orden
2 | SELECT 3001, REF(C), SYSDATE, '30-MAY-1999', lineas_pedido_t(), N
3 | FROM cliente_tab C
4 | WHERE C.clinum= 3 ;
5 |
6 | INSERT INTO THE ( --selecciona el atributo pedido de la orden
7 | SELECT P.pedido
8 | FROM ordenes_tab P
9 | WHERE P.ordnum = 3001
10 | )
11 | VALUES (linea_t(30, NULL, 18, 30));
12 | --inserta una linea de pedido anidada
```

Para poner condiciones a las tuplas de una tabla anidada, se pueden

utilizar cursores dentro de un SELECT o desde un programa PL/SQL. Veamos aquí un ejemplo de acceso con cursores. Utilizando un ejemplo anterior, vamos a recuperar el número de las ordenes, sus fechas de pedido y las líneas de pedido que se refieran al ítem 'CH4P3'.

```
1 | SELECT ord.ordnum, ord.fechpedido,  
2 | CURSOR (SELECT * FROM TABLE(ord.pedido) lp WHERE lp.item= 'CH4P3  
3 | FROM ordenes_tab ord;
```

La cláusula THE también sirve para seleccionar las tuplas de una tabla anidada. La sintaxis es como sigue:

```
1 | SELECT ... FROM THE (subconsulta) WHERE ...
```

Por ejemplo, para seleccionar las primeras dos líneas de pedido de la orden 8778 se hace:

```
1 | SELECT lp FROM THE  
2 | (SELECT ord.pedido FROM ordenes_tab ord WHERE ord.ordnum= 8778)  
3 | WHERE lp.linum<3;
```



Autoevaluación

Señala la opción incorrecta.

- ☐ SELECT nombre FROM ptab1;
- ☐ SELECT c1.nombre FROM ptab2;
- ☐ SELECT p.c1.nombre FROM ptab2 p;
- ☐ SELECT p.c1.nombre FROM ptab3 p;

No es la respuesta correcta, prueba de nuevo.

Muy bien, vamos por buen camino.

No es correcto.

No es correcto.

Solución

1. Incorrecto (#answer-87_63)
2. Opción correcta (#answer-87_90)
3. Incorrecto (#answer-87_93)
4. Incorrecto (#answer-87_96)

8.- Gestión de transacciones en las BDOR..



Caso práctico

—¡Por fin ha llegado el viernes! —dice **Ana** a **Juan**, y continúa— después de estas dos últimas semanas revisando sistemas objeto-relacionales ¡he terminado agotada!, pero el esfuerzo me ha compensado. Ahora tengo mucho más claro, las diferencias entre una base de objetos y una objeto-relacional. Pero... —se queda pensativa por un instante y entonces continúa— las transacciones ¿se gestionan de manera similar en ambos tipos de sistemas?



Q Ministerio de Educación (Uso educativo [nc](#))

Juan le responde, —justo es eso lo que nos queda por ver, **Ana**. ¿Empezamos hoy o lo dejamos para el lunes?

Ana sonríe y le dice —Creo que **Ada** viene para acá, ¿no oyes sus pasos?

Como en cualquier otro Sistema de Bases de datos, en un Sistema de bases de objetos u objeto relacional, **una transacción** es un conjunto de sentencias que se ejecutan formando una unidad de trabajo, esto es, en forma indivisible o atómica, o se ejecutan todas o no se ejecuta ninguna.

Mediante la gestión de transacciones, los sistemas gestores proporcionan un acceso concurrente a los datos almacenados, mantienen la integridad y seguridad de los datos, y proporcionan un mecanismo de recuperación de la base de datos ante fallos.

Un ejemplo habitual para motivar la necesidad de transacciones es el traspaso de una cantidad de dinero (digamos 10000€) entre dos cuentas bancarias. Normalmente se realiza mediante dos operaciones distintas, una en la que se decrementa el saldo de la cuenta origen y otra en la que incrementamos el saldo de la cuenta destino.

Para garantizar la integridad del sistema (es decir, para que no aparezca o desaparezca dinero), las dos operaciones tienen que completarse por completo, o anularse íntegramente en caso de que una de ellas falle.

Las transacciones deben cumplir el criterio ACID.

- **Atomicidad**. Se deben cumplir todas las operaciones de la transacción o no se cumple ninguna; no puede quedar a medias.
- **Consistencia**. La transacción solo termina si la base de datos queda en un estado consistente.
- **Isolation** (Aislamiento). Las transacciones sobre la misma información deben ser independientes, para que no interfieran sus operaciones y no se

produzca ningún tipo de error.

- **Durabilidad.** Cuando la transacción termina el resultado de la misma perdura, y no se puede deshacer aunque falle el sistema.

Algunos sistemas proporcionan también **puntos de salvaguarda** (savepoints) que permiten descartar selectivamente partes de la transacción, justo antes de acometer el resto. Así, después de definir un punto como punto de salvaguarda, puede retrocederse al mismo. Entonces se descartan todos los cambios hechos por la transacción después del punto de salvaguarda, pero se mantienen todos los anteriores.

Originariamente, los puntos de salvaguarda fueron una aportación del estándar SQL99 de las Bases de Datos Objeto-Relacionales. Pero con el tiempo, se han ido incorporando también a muchas Bases de Datos Relaciones como MySQL (al menos cuando se utiliza la tecnología de almacenamiento InnoDB).

Anexo I.- Estándar ODMG-93 u ODMG.

El **estándar ODMG** (Object Database Management Group) trata de estandarizar conceptos fundamentales de los Sistemas Gestores de Bases de Datos Orientados a Objetos (SGBDOO) e intenta definir un **SGBDOO como un sistema que integra las capacidades de las bases de datos con las capacidades de los lenguajes de programación orientados a objetos**, de manera que los objetos de la base de datos aparezcan como objetos del lenguaje de programación.

Fué desarrollado entre los años 1993 y 1994 por representantes de un amplio conjunto de empresas relacionadas con el desarrollo de software y sistemas orientados a objetos.

1. Arquitectura del estándar ODMG

La arquitectura propuesta por ODMG consta de:

- Un **modelo de objetos** que permite que tanto los diseños, como las implementaciones, sean portables entre los sistemas que lo soportan.
- Un **sistema de gestión** que soporta un lenguaje de bases de datos orientado a objetos, con una sintaxis similar a un lenguaje de programación también orientado a objetos.
- Un **lenguaje de base de datos** que es especificado mediante:
 - Un Lenguaje de Definición de Objetos (ODL)
 - Un Lenguaje de Manipulación de Objetos (OML)
 - Un Lenguaje de Consulta (OQL)

siendo todos ellos portables a otros sistemas con el fin de conseguir la portabilidad de la aplicación completa.

- Enlaces con lenguajes Orientados a Objetos como C++, Java, Smaltalk.

El **modelo de objeto ODMG** es el modelo de datos en el que están basados el ODL y el OQL. Este modelo de objeto proporciona los tipos de datos, los constructores de tipos y otros conceptos que pueden utilizarse en el ODL para especificar el esquema de la base de datos de objetos.

Vamos a destacar algunas de las **características más relevantes** del estándar ODMG:

- Las primitivas básicas de modelado son los **objetos** y los **literales**.
- Un objeto tiene un **Identificador de Objeto** (OID) y un estado (valor actual) que puede cambiar y tener una estructura compleja. Un literal no tiene OID, pero sí un valor actual, que es constante.
- El estado está definido por los valores que el objeto toma para un conjunto de propiedades. Una propiedad puede ser:
 - Un atributo del objeto.

- Una interrelación entre el objeto y otro u otros objetos.
- Objetos y literales están organizados en **tipos**. Todos los objetos y literales de un mismo tipo tienen un comportamiento y estado común.
- Un objeto queda descrito por cuatro características: identificador, nombre, tiempo de vida y estructura.
- Los tipos de objetos se descomponen en atómicos, colecciones y tipos estructurados.
 - **Tipos atómicos** o básicos: constan de un único elemento o valor, como un entero.
 - **Tipos estructurados**: compuestos por un número fijo de elementos que pueden ser de distinto tipo, como por ejemplo una fecha.
 - **Tipos colección**: número variable de elementos del mismo tipo. Entre ellos:
 - **Set**: grupo desordenado de elementos y sin duplicados.
 - **Bag**: grupo desordenado de elementos que permite duplicados.
 - **List**: grupo ordenado de elementos que permite duplicados.
 - **Array**: grupo ordenado de elementos que permite el acceso por posición.

Algunos fabricantes sólo ofrecen vinculaciones de lenguajes específicos, sin ofrecer capacidades completas de ODL y OQL.